

stk(super tool kit)

STK是一个工具箱，包含一系列芯片设计和验证的工具，提高工作效率。

序号	命令	说明
1	bsim	build and simulation，封装VCS的编译和仿真命令，实现简单命令完成编译和仿真
2	genCRG	生成CRG模型，和CRG RTL
3	genEnv	根据接口列表生成验证环境
4	genExcel	生成excel demo文件，支持接口列表，寄存器表单，crg表单，bus表单
5	genReg	根据excel表生成UVM寄存器模型，和CRG RTL代码
6	instRTL	生成verilog和vhdl的例化，用于代码集成和环境搭建
7	parseLog	解析编译或者仿真log，用于判断仿真或者回归的是否通过
8	regress	验证回归工具，自动化用例提交和结果判断，支持数据库和个性化窗口显示
9	regrServer	读取regress生成的数据库，在浏览器中以各种图表的形式显示回归数据。
10	genDummy	生成RTL的dummy文件
11	instConn	soc集成多个子模块，生成顶层文件
12	extFList	filelist中使用宏的情况下，根据宏定义提取filelist
13	yaml扩展	yaml语法扩展

0. 安装方法

0.1 linux

1. 从github下载linux版本stk工具
2. 解压stk到相应目录
3. 把路径添加Path中
 - bash: .bashrc中添加`export PATH=$PATH:/path/to/stk`；执行`source .bashrc`
 - csh: .cshrc中添加`setenv PATH $PATH:/path/to/stk`；执行`source .cshrc`

0.2 windows

1. 从github下载windows版本stk工具
2. 解压到相应目录
3. 把路径添加path环境变量中
4. 在cmd中执行`stk`

1. bsim

1.1 说明

bsim是用于方便VCS编译和仿真的一个工具，VCS在编译和仿真过程中需要添加众多参数选项，为了减少命令行参选项输入和管理各个选项，bsim工具通过yaml文件清晰呈现选项，并封装功能块为特定选项，方便使用。

bsim的相比其他脚本，有如下特点：

1. 使用yaml, (支持include,引用，继承等功能)方便选项的管理和扩展，同时支持选项对其覆盖，配置灵活。
2. 将VCS的常用功能做成了开关选项，减少选项输入，封装的功能块有
 1. partcmp
 2. xprop
 3. initreg
 4. ccov
 5. fsdb
 6. gui
3. 提供3种触发编译的模式，（默认/-b/-u）,用户只需要执行sim命令即可完成编译和仿真，非常方便。

1.2. 使用

1. 只仿真

```
stk bsim                                # simulation only,
```

如果仿真的时候发现没有生成simv，会自动进行编译

2. 编译和仿真

bsim使用分为2步法和3步法：

• 2步法：

```
stk bsim -step2 -b                      # compile+simulation,
stk bsim -step2 -b -ns                  # compile only,
```

• 3步法：

```
stk bsim -step3 -a -ns                  # analysis(all),
stk bsim -step3 -a -b -ns              # analysis(all)+elaboration,
stk bsim -step3 -a                     # analysis(all)+elaboration+simulation,
stk bsim -step3 -a -b                  # analysis(all)+elaboration+simulation,
stk bsim -step3 -b -ns                  # elaboration
stk bsim -step3 -b                     # elaboration+simulation,
stk bsim -step3 -i 2                    # analysis(only
flist2)+elaboration+simulation(recommend),
stk bsim -step3 -i 2 -ns                # analysis(only flist2)
```

执行analysis之前需要一个synopsis_sim.setup的文件，程序会根据flists配置中的lib信息，生成一个synopsis_sim.setup文件，存放在work_dir/exec目录下进行使用。如果需要修改synopsis_sim.setup中的内容，可以添加命令到ana.prev_cmd中，修改synopsis_sim.setup

调用使用-u选项判断file list更新与否是与前一次使用-u相比的。即更新检查依赖于上一次的更新检查的结果，会有如下特殊情况。

1. 第一次使用-u选项，必然会认定有更新
2. 调用没有用-u选项命令对有更新的filelist进行编译后，调用使用-u选项命令还是会被认定为有更新，而导致重新编译，因为相对上一次使用-u选项，确实有更新，虽然这个更新已经被编译过，但是没有用-u选项，没有被记录下来。
3. 使用-u选项要有连贯性，要么一直自己跟踪是否有更新，通过默认方式或者-b方式来编译，要么一直使用-u选项

1.3. 配置(yaml和选项)

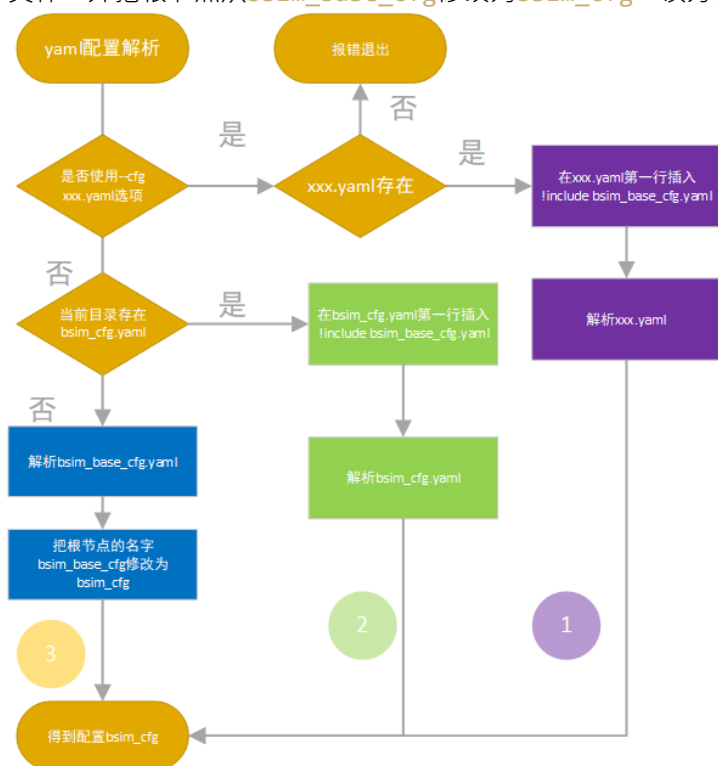
bsim的配置来源分为2部分，yaml配置文件和bsim的选项

1.3.1 yaml

bsim使用的yaml文件相对于标准的yaml文件进行了特性扩展，扩展支持:include、继承、引用、引用环境变量等功能，语法和使用方式参考[yaml语法扩展](#)

bsim获取yaml配置分为3种情况：

1. 用--cfg xxx.yaml指定了配置文件，且文件存在，那么解析该xxx.yaml，xxx.yaml根节点必须命名为bsim_cfg
2. 没有指定了配置文件，当前目录存在bsim_cfg.yaml文件，那么解析该bsim_cfg.yaml，bsim_cfg.yaml根节点必须命名为bsim_cfg
3. 即没有指定配置文件，当前目录也不存在bsim_cfg.yaml文件，那么解析stk提供的bsim_base_cfg.yaml配置文件，并把根节点从bsim_base_cfg修改为bsim_cfg，该方式使用stk的默认配置，适用于临时使用。



3中方式中，推荐使用第二种，即在仿真目录下存放一个bsim_cfg.yaml文件，这样仿真自动会使用该配置文件。

方式1和方式2都会默认在配置文件头插入!include bsim_base_cfg.yaml语句，这样用户编写bsim_cfg的时候可以直接继承bsim_base_cfg，进行扩展和修改，减少配置文件编辑量。

配置文件根节点必须为**bsim_cfg**，包含4个子节点，分别为：ana,eac,sim,com,

- ana用于ana命令
- eac用于elab和cmp命令
- sim用于sim命令
- com表示common，即ana/elab/cmp/sim都会使用该部分配置

1.3.1.1 ana

- vlog_opts：verilog analysis添加的选项
- vhdl_opts：vhdl analysis添加的选项
- prev_cmd：执行analysis前执行的命令，值可以是一条命令，也可以是一个list，包含多个命令
- post_cmd：执行analysis后执行的命令，值可以是一条命令，也可以是一个list，包含多个命令

```
ana:
  prev_cmd: pwd
  post_cmd:
    - cmd0
    - cmd1
```

1.3.1.2 eac

- opts: 编译添加的选项
- ccov_opts：如果com.ccov选项打开时，编译添加的选项
- top_hire：验证环境或者设计的顶层名字
- partcmp：分块编译的开关
- partcmp_mode：分块编译的模式，可以选择VCS提供的模式：adaptive_sched/autopart/autopart_low，也可以编写topcfg.v文件来收到设计分块。
- xprop：xprop选项开关
- xprop_mode：xprop的模式，可以选VCS提供的3个模式：vmerge/tmerge/xmerge，也可以选择编写xprop.cfg配置文件
- initreg：initreg选项开关
- initreg_cfg：initreg设置的配置文件
- prev_cmd：执行elaborate和compile前执行的命令，值可以是一条命令，也可以是一个list，包含多个命令
- post_cmd：执行elaborate和compile后执行的命令，值可以是一条命令，也可以是一个list，包含多个命令

1.3.1.3 sim

- opts：仿真添加的选项
- ccov_opts：com.ccov选项打开时，仿真添加的选项
- tc：设置UVM的执行用例名
- plusarg：如果仿真包含众多plusarg参数，可以将这些参数统一加入到一个文件中，然后将文件添加此处。这用户参数与VCS的选项分离，方便管理。
- rand_seed：是否产生随机种子进行仿真，通常在用例仿真稳定后才开启该选项。
- step3：VCS编译的模式，默认情况下，sim命令引发编译会采用2步法（compile），step3选项开启后，sim命令引发编译会采用3步法（analysis+elaborate）
- prev_cmd：执行simulation前执行的命令，值可以是一条命令，也可以是一个list，包含多个命令
- post_cmd：执行simulation后执行的命令，值可以是一条命令，也可以是一个list，包含多个命令

- `print_level`: UVM_NONE, UVM_LOW, UVM_MEDIUM, UVM_HIGH, UVM_FULL, UVM_DEBUG

1.3.1.4 com

- `flists`: file list, 该配置除了写file list文件, 还要说明file list的文件类型, 编译的lib存放在那个逻辑库, 多个file list可以写成列表, 格式为: `type|lib|filelist`, 即3部分内容通过"|"分隔,
 - `type`: 可选值为: `vlog`和`vhdl`, 分别表示filelist的文件为verilog文件和vhdl文件
 - `lib`: analysis存放结果的位置, 如果省略存放在work库中
 - `filelist`: 即file list文件

三部分内容中, `type`和`lib`可以省略, 有如下三中情况:

- `type|lib|filelist`: 不省略
- `type|filelist`: 省略lib, 默认为work
- `filelist`: 省略lib, 默认为work, 省略type, 默认为vlog

不省略lib, 则type也不能省略

例子:

```
# 一个file list
flists: $$env_dir/tb/tb.f # 等效于:vlog|work|$$env_dir/tb/tb.f
# 多个file list
flists:
- vhdl|vh|$$env_dir/tb/vh.f
- vlog|$$env_dir/tb/abc.f # 等效于:vlog|work|$$env_dir/tb/abc.f
- $$env_dir/tb/env.f # 等效于:vlog|work|$$env_dir/tb/env.f
```

多个filelist通常有依赖关系, 要把被依赖的filelist写在前面

- `flist_define`: 解析filelist时需要的宏
- `work_dir`: 工作目录, 可以为相对目录, 也可以为绝对目录。该目录下会创建: `exec/wave/log/verdi/cov/merge`等目录, 用于存放编译和仿真的文件。其中波形存放在wave目录下, log文件存放在log目录下, 覆盖率文件存放在cov目录下, 调用merge命令, 得到merge覆盖率存放在merge目录下。
- `ccov`: 覆盖率开关, 会同时影响编译和仿真
- `fsdb`: dump波形的开关, 使能该开关会后无效在代码中调用`$fsdbDumpvars`, bsim会生成tcl脚本自动dump波形。
- `uvm_en`: UVM使能开关, 这个为True, 编译时候才会编译UVM
- `uvm_work`: UVM analysis时, lib存放位置
- `prev_cmd`: 执行bsim前执行的命令
- `post_cmd`: 执行bsim后执行的命令

简单例子:

```

bsim_cfg(bsim_base_cfg):
  com:
    flists:
      - vhdl|work|$$env_dir/tb/vh.f
      - vlog|work|$$env_dir/tb/abc.f
    fsdb: true
  sim:
    opts+: >-
      +uvm_set_verbosity="\*",connect_ph,UVM_HIGH,build\"

```

1.4.1.5 bsim_base_cfg.ymal

```

bsim_base_cfg:
  ana:
    prev_cmd:
    post_cmd:
    vlog_opts: >-
      -full64 -ntb_opts uvm-1.2 -sverilog -lca # 如果要添加"需要使用反斜杠\"
      -timescale=1ns/1ps
      -ntb_opts check
      +vcs+lic+wait
      -kdb
    vhdl_opts: >-
      -full64 -vhdl93 -kdb

  eac: # elaborate and compile
    prev_cmd:
    post_cmd:
    opts: >-
      -full64 -ntb_opts uvm-1.2 -sverilog -lca
      -timescale=1ns/1ps
      -ntb_opts check
      +vcs+lic+wait
      -kdb +lint=TFIPC-L
      +error+100
      # -P $$VERDI_HOME/share/PLI/VCS/LINUX64/novas.tab
      # $$VERDI_HOME/share/PLI/VCS/LINUX64/pli.a
    ccov_opts: -cm line+fsm+cond+tgl+assert+branch -cm_tgl mda
    top_hire: top
    partcmp: false
    partcmp_mode: adaptive_sched #adaptive_sched | autopart | autopart_low |
    autopart_high | $(ENV_SIM_DIR)/cfg/topcfg.v
    xprop: false
    xprop_mode: # vmerge | tmerge | xmerge | $(ENV_SIM_DIR)/cfg/xprop.cfg
    initreg: false
    initreg_cfg:

  sim:
    prev_cmd:
    post_cmd:
    opts: >-
      -full64 +notimingcheck -assert nopostproc +vcs+lic+wait

```

```

-assert global_finish_maxfail=10
+ntb_stop_on_constraint_solver_error=1
+uvm_set_action="*,RegModel,UVM_WARNING,UVM_NO_ACTION"
ccov_opts: -cm line+fsm+cond+tgl+assert+branch -cm_tgl mda
tc: tc001_sanity
plusarg: # $(ENV_SIM_DIR)/cfg/args/default.cfg
rand_seed: True
step3: True

com:
  flists: # ana+elab
  flist_defines: DUMMY_DDR+DUMMY_PCIE
  work_dir: work # ana+elab+sim
  ccov: false # elab+sim
  fsdb: false # elab+sim
  uvm_work: work
  uvm_en: True
  prev_cmd:
  post_cmd:

```

1.3.2 选项

bsim包含很多选项，选择在analysis/elaborate/simulate的不同阶段起作用，把选项分为了多个组

命令	选项组成
common	在所有流程中会可能用到的选项
elab+cmp	elab和cmp可能用到的选项
elab+cmp+sim	elab/cmp和sim可能用到的选项
sim	sim可能用到的选项
action	功能选项

1.3.2.1 common

- -cfg：指定配置的yaml文件，如果不指定会尝试查看当前目录下是否有bsim_cfg.yaml的文件，有的话使用该文件，如果没有则使用默认的bsim_base_cfg.yaml。
- -d/-work_dir：工作目录，该目录下会创建exec/wave/log/cov/merge/verdi等目录，存放编译和仿真的文件。使用该选项会覆盖掉yaml的work_dir配置。默认认为work目录
- 添加选项到执行命令中：-co='+define+ABC'
 - -ao: 添加verilog的analysis选项
 - -vao: 添加vhdl的analysis选项
 - -co: 添加compiled的选项
 - -eo: 添加elaborate的选项
 - -so: 添加simulation的选项
- -f：设置file list，与yaml中的flists格式一样，分为3部分：type|lib|filelist，其中type,lib可以省略。如果有多个filelist可以通过"+"连接起来。比如：bsim ana --flists='vhdl|xyz|xyz.f+abc.f'，analysis的顺序是连接的顺序。使用该选项会覆盖yaml中flists配置

由于|会被linux识别为管道符，需要flists的值需要用单引号(')，或者双引号包裹("")

- -t: 追加yaml中预先写好的选项到命令中, 比如: `bsim ana --tag=abc`, 那么会将bsim_cfg.ana.abc的值追加到ana命令中。命令只能取特定节点下的tag, 具体参考3.3.4 yaml和选项协同配置(tag)
- -uvm: uvm使能, 使用该选项, 在ana的时候, 会独立analysis UVM, 同时在cmp和elab的时候添加-ntb_opts uvm-1.2选项。使用该选项会使能uvm, 无论yaml中的uvm_en值是True还是False
- -D: 设置解析filelist的宏, 具体参考stk.extFList
- -u: 设置触发编译的条件为filelist更新, 选项与-b互斥。如果为2步法, 那么有filelist更新, 就会进行compile;如果为3步法, 会先对有更新的filelist进行analysis, 然后elaborate。
- -debug: 测试模式
- -dump_cfg: 把bsim配置dump到work_dir目录下

1.3.2.2 elab_cmp

- -partcmp: 分块编译开关, 使用该选项, 会在compile和elaborate时使用分块编译, 分块模式有--partcmp_mode设置。使用该选项会使能分块编译, 无论yaml中的partcmp值是True还是False
- --partcmp_mode: 分块编译的模式, 可以选择VCS提供的模式: adaptive_sched/autopart/autopart_low, 也可以编写topcfg.v文件来收到设计分块。使用该选项会覆盖yaml中partcmp_mode配置
- -xprop: xprop开关, 使用该选项会使能xprop, 无论yaml中的xprop值是True还是False
- -xprop_mode: xprop的模式, 可以选VCS提供的3个模式: vmerge/tmerge/xmerge, 也可以选择编写xprop.cfg配置文件, 使用该选项会覆盖yaml中xprop_mode配置

1.3.2.3 elab_cmp_sim

- -ccov: 覆盖率收集开关, 使用该选项会使能覆盖率收集, 无论yaml中的ccov值是True还是False
- -ccov_opts: 覆盖率收集选项。覆盖率收集选项在compile/elaborate和simulation都需要设置, 且选项不完全一样。
- -initreg: initreg开关, 使用该选项会使能initreg, 无论yaml中的initreg值是True还是False
- -initreg_cfg: 设置initreg的配置文件
- -top: 设置top模块
- -gui: 启动图形界面
- -fsdb: 波形dump开关, 使能该开关会后无需在代码中调用\$fsdbDumpvars, bsim会生成tcl脚本自动dump波形。使用该选项会dump波形, 无论yaml中的fsdb值是True还是False

默认-fsdb会dump top开始的所有层次, 如果想修改层次, 可以手动编写fsdb.do文件, 让后在prev_cmd中把编写的文件拷贝到work_dir/exec目录下, bsim会自动执行该文件。

1.3.2.4 sim_only

- -seed: 设置simulation的种子号
- -p: 如果仿真包含众多plusarg参数, 可以将这些参数统一加入到一个文件中, 然后将文件添加此处。这用户参数与VCS的选项分离, 方便管理。使用该选项会覆盖yaml中plugarg配置
- -pl: print_level设置UVM的打印等级, 可选值为: UVM_NONE, UVM_LOW, UVM_MEDIUM, UVM_HIGH, UVM_FULL, UVM_DEBUG
- -rand_seed: 种子随机开关, 使用该选项会使能分块编译, 无论yaml中的rand_seed值是True还是False
- -step3: 3步法开关, 使用该选项会使能分块编译, 无论yaml中的step3值是True还是False
- -step2: 2步法开关
- -tc: 设置UVM的执行用例名使用该选项会覆盖yaml中tc配置
- -pname: 为仿真log文件名添加额外的标识

1.3.2.5 action

- **-b**：强制触发编译，选项与**-u**互斥。3步法时，没有结合**-i**选项时，会强制analysis所以filelist，然后elaborate；如果结合**-i**使用，则只强制analysis -i指定的filelist，然后在elaborate。2步法时，是否使用**-i**，行为都一样，即强制compile。
- **-i**：
 - 不与**-u**结合时：一定编译-i指定的file list
 - 与**-u**结合时：只查看-i指定的file list是否有更新

1.3.3 yaml与选项关系

1.3.3.1 yaml与选项共同配置

yaml和程序选项都可以控制以下配置，其中bsim选项的优先级更高，只有bsim选项没有配置的情况下，才会采用ymal的配置值

- **bool类型**：
`value = bsim.value or yaml.value`：比如`stk bsim -fsdb`，那么dump fsdb功能打开，无论yaml中配置的fsdb为True还是false
- **str类型**：
 - bsim添加添加参数：`value = bsim.value`，此时会覆盖yaml的配置值，比如
`yaml.com.work_dir=work, stk bsim -work_dir abc`，那么work_dir的配置值为abc
 - bsim不添加参数：`value = yaml.value`

yaml配置	bsim选项	类型
yaml.com.flists	-f	str(file list)
yaml.com.flist_defines	-D	str(path)
yaml.com.work_dir	-work_dir/-d	str(path)
yaml.com.ccov	-ccov	bool
yaml.com.fsdb	-fsdb	bool
yaml.com.uvm_en	-uvm	bool
yaml.eac.ccov_opts	-ccov_opts	str(options)
yaml.eac.initreg	-initreg	bool
yaml.eac.initreg_cfg	-initreg_cfg	str(configure file)
yaml.eac.partcmp	-partcmp	bool
yaml.eac.partcmp_mode	-partcmp_mode	str(mode or configure file)
yaml.eac.xprop	-xprop	bool
yaml.eac.xprop_mode	-xprop_mode	str(mode or configure file)
yaml.eac.top_hire	-top	str(hire)
yaml.sim.step3	-step3	bool
yaml.sim.plugarg	-plugarg	file
yaml.sim.tc	-tc	str(test case name)

yaml配置	bsim选项	类型
yaml.sim.rand_seed	-rand_seed	bool
yaml.sim.ccov_opts	-ccov_opts	str(options)
yaml.sim.print_level	-pl	str(options)

ccov_opts选项，会同时影响编译的ccov_opts和仿真的ccov_opts，sim命令使用该选项时，不能触发编译。

1.3.3.2 yaml独立配置

下面这些yaml配置不受选项控制

- ana.vlog_opts
- ana.vhdl_opts
- elab.opts
- sim.opts
- com.uvm_work

1.3.3.3 选项独立配置

下面这些选项与yaml配置无关

- -ao/vao
- -co/-eo
- -so
- -b/-u/-i
- -gui
- -seed

1.3.3.4 yaml和选项协同配置(tag)

在yaml添加tag(就是字典)，并通过-t选项来引用，将tag的值添加到执行的命令中，从而减少命令行输入的字符数量。

- yaml在ana/elab/sim下面添加tag_name: tag_options,
- 在选项中使用cmd -t tag_name, -t可以多次使用，且一次可以添加多个tag_name,比如sim -t tag_name1+tag_name2 -t tag_name3,

举例：

yaml:

```
bsim_cfg:
  eac:
    debug: -debug_access+all
```

命令：

```
stk bsim -t debug
```



通过 `-t debug` 往命令中添加了 `-debug_access+all` 选项，这种方式在添加的选项比较长的时候比较有优势，如果短可以直接使用 `-ao/co/eo/so` 选项

注意：tag 引用节点位置与命令有对应关系。

命令	可以引用的yaml节点
ana	bsim_cfg.ana/bsim_cfg.com
elab	bsim_cfg.eac/bsim_cfg.com
cmp	bsim_cfg.eac/bsim_cfg.com
sim	bsim_cfg.sim/bsim_cfg.com

1.4. 规则

1.4.1 log名字和路径

路径：所以log都存放在work_dir/log目录下
名字：

- 1. analysis的log文件名与filelist有关，logname={filelist_name}{filelist_idx}_an.log
- 2. compile的log文件名固定为cmp.log
- 3. elaborate的log文件名固定为elab.log
- 4. simulation的log文件名与tc,seed,pname值有关，logname={tc_name}{pname}{seed}.log，seed默认为12345678

2. genCRG

生成CRG的比对模型和CRG的RTL代码

2.1 使用

-- 待开发 --

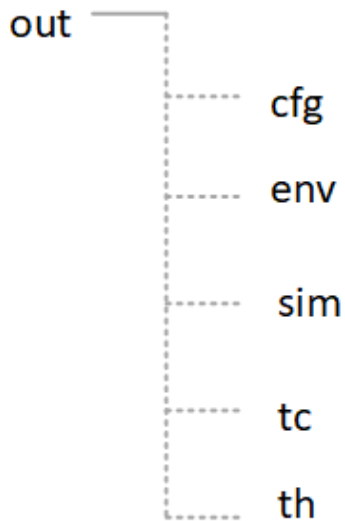
3. genEnv

根据excel的接口列表生成验证环境，接口列表的填写和格式参考genExcel

3.1. 使用

命令

- 1. `stk genEnv xxx.excel --common common.yaml`：生成基于UVM的验证环境
 - 2. `stk genEnv -s xxx.excel --common common.yaml`：生成基于SVK的验证环境
- 将会在当前目录下生成一个out文件夹，文件夹内为生成的环境，验证环境的文件结构如下



- `cfg` : 放filelist等配置文件
- `env` : 放env、agent、rm等
- `sim` : 仿真目录，包含一个sourceme文件
- `tc` : 测试用例目录
- `th` : 验证环境的接口连接和harness相关文件

common.yaml需要包含如下内容：

- `author` : 用户名，用于显示在生成文件的文件头中
- `email` : 邮箱地址，用于显示在生成文件的文件头中
- `proj` : 项目名，用于设置生成文件的名称，比env的名称
- `top` : 要测试模块的module名字，用于生成module例化
- `prefix` : 前缀，用于生成文件名，比如prefix为svk，生成的文件名为svk_kirin_env.sv

```
author: Jakod
email: JakodYuan@outlook.com
prefix: ''
proj: kirin
top: gpu
```

填写规范:				
1. "汇总"页签包含所以接口列表, 点击接口链接可以跳转到相应接口页签				
2. 页签名为接口名字, 接口名尽量短, 减少agent名字长度				
3. work_mode为接口的工作模式, 可以填: MASTER/SLAVE/MONITOR				
4. B1为接口所在RTL模块的名字				
5. hierarchy为接口在RTL中的例化层次, 从top/harness开始				
6. 点击汇总链接可以跳转到"汇总"页签				
7. type:端口类型, 可为: signal/interface/标准接口(AXI/AHB/APB,ACE)				
8. type=signal, 分别填direction/type/name/description				
9. type=interface, 分别填type/name/description, direction可以填NA				
10. type=标准接口, 分别填prefix/suffix/attr/description				
11. type=标准接口, prefix:填相对标准名字多出的前, suffix:填相对标准名字多出的后缀, attr(data/addr/id):填接口的配置信息				
12. type=标准接口, 目前支持AXI/ACE/APB/AHB				
13. type=signal和type=itf可以在一个表中, type=标准接口, 必须独立为一个表, 且一个表只有一个接口				
work_mode	module	hierarchy		汇总
MASTER	gpu	top.u_gpu		
mode	direction	type	name	description
signal	out	logic [31:0]	addr	address
struct	out	task_type	data	data
interface	NA	svt_axi_if	axi_mst	master axi interface
work_mode	module	hierarchy		汇总
SLAVE	cpu	top.u_gpu.u_cpu		
type	prefix	suffix	attr	description
AXI	axi_s_		addr_width=128 data_width=32 id_width=16 osd_num=8	slave axi interface
work_mode	module	hierarchy		汇总
MASTER	cpu	top.u_gpu.u_cpu		
type	prefix	suffix	attr	description
AHB	ahb_m_		addr_width=128 data_width=32	master ahb interface
work_mode	module	hierarchy		汇总
SLAVE	cpu	top.u_gpu.u_cpu		
type	prefix	suffix	attr	description
APB	apb_s_		addr_width=128 data_width=32	slave apb interface

3.2. 环境的使用与修改

生成的验证环境可以通过`stk bsim`直接运行

进入: `sim`目录, 执行`source sourceme;stk bsim`, 即可进行编译和仿真

由于agent还是空壳, 例化的module是假的, 所以达到真实的测试环境, 还需要对环境进行修改。

1. 在`cfg/tb.f`中添加RTL的filelist, 且要放在`harness.sv`前面, 当然RTL的filelist也可以独立成一个filelist
2. 注释掉`cfg/tb.f`中的`top.sv`, 这是一个假的被测module, 用RTL中的真实module替代
3. 完善agent中的driver
4. 解注释interface的链接, 如果使用的是基于UVM的环境, 这部分内容在`/th/harness.sv`中。如果使用的是基于SVK的验证环境, 这部分内容在`/th/xx_bind.sv`文件中。
5. 在`sim`目录下执行`stk bsim -b -fsdb`即可完成编译和仿真。

4. genExcel

生成excel模板

4.1. 使用

- 命令: `stk genExcel -m [reg/itf/crg/bus] xxx.xlsx`
- 参数: `-m`: 指定生成模板的类型, 可选值: `reg/itf/crg/bus`
- 输出: 在当前目录下, 生成命令中命名的excel文件

4.2. reg格式

1	Module Name	Owner	Data Width	Address Width	Base Address	Bus Type	返回汇总
2	demo		32	64	0x0	APB	
3	Register/Group Name	Address Offset	Field Bits	Field Name	Reset Value	Access	Field Discription
4	TEST0	0x80000000	17:2	pll	0x0123	RW	pll配置
5			1:1	soft_reset	0x0	RW	软复位:0:复位, 1:解复位
6			0	clk_gate	0x0	RW	时钟关断, 0:开启时钟, 1:关闭时钟
7	TEST1	0x80000010	1:1	data1_1	0x0	RO	值
8			0:0	data1_0	0x0	RO	值
9	TEST2	0x80000018	1:1	data2_1	0x0	RC	值
10			0:0	data2_0	0x0	RC	值
11	TEST3	0x80000020	0:0	data3	0x0	WC	值
12	TEST4	0x80000028	0:0	data4	0x0	W1C	值
13	TEST5	0x80000030	31:0	bit_mask	0x0	BMW	数据
14	TEST6	0x80000038	0:0	data6	0x0	W1P	值
15	ARRAY	0x80000040*10	15:0	coeff0	0xffff	CYC_CNT_RC	系数
16			31:16	coeff1	0xffff	CYC_CNT_RC	系数
17	TEST7	0x800000A0	0:0	data7	0x0	SAT_CYC_WC	值
18	TEST8	0x800000A4	31:0	bit_mask	0x0	CYC_CNT_WC	数据
19	SRC	0x800000A8	31:0	src	0x0	RO	
20	RAW	0x800000AC	31:0	raw	0x0	RO	
21	CLEAR	0x800000B0	31:0	clear	0x0	W1P	
22	MASK	0x800000B4	31:0	mask	0x0	RW	
23	STATUS	0x800000B8	31:0	status	0x0	RO	
24	ADDR_MSB	0x800000C0*10*0x8	31:0	addr_msb	0x0	RW	
25	ADDR_LSB	0x800000C4*10*0x8	31:0	addr_lsb	0x0	RW	
26	Reg Group Name NOT_SAME	0x80000110	31:16	fro	0x0	RO	
27			15:8	frw	0x0	RW	
28			7:0	fwo	0x0	WO	
29	COMMON	0x80000120	31:0	status	0x1	INT.STATUS	
30	COMMON	0x80000124	31:0	mask	0x2	INT.MASK	
31	COMMON	0x80000128	31:1	clear	0x3	INT.CLEAR	

1. **Module Name** : 填写模块名字, 生成的文件会使用该名字
2. **Data Width** : 必须为10进制数, 且为8的倍数
3. **Addr Width** : 必须为10进制数
4. **Base Addr** : 必须为16进制数, 以0x开头
5. **Bus Type** : 支持APB/AHB/AXI_LITE
6. **Register/Group Name** : 这个名字可以是Reg Group name或者reg name, 推荐使用大写, 不同寄存器和寄存器组的名字不能重复, 同一个寄存器组的Group name相同。
7. **Address Offset** : 有3中填写形式, addr/addrtimes/addrtimes*stride
 - **addr** : 0x开头的16进制值, 比如0x80000
 - **addr*times** : 在7.1的基础上添加一个例化次数值times, 表示该寄存器从addr开始, 例化times次, 每例化一次后, 下一次例化会紧挨上一次的地址例化, 比如: 0x8000*10, 寄存器地址依次为0x8000, 0x8000+(data_width/8)*1...0x8000+(data_width/8)*9
 - **addr*times*stride** : 在7.2的基础上添加一个stride, stride为0x开头的16进制数, 比如0x10, 9.2的stride默认为 (data_width/8), 这里可以添加stride实现下一轮例化的地址与上一轮地址之间留空。用于某些寄存器地址有对齐的需求。stride>=data_width/8)
8. **Field Bits** : field的bit范围, 多bit域, 域的格式为MBS:LBS (7:0), 单bit域可以用MBS:LBS (0:0) 格式, 也可以直接写bit(0)的索引。
9. **Field Name** : 寄存器域名推荐用小写, 不同寄存器的field名字可以相同
10. **Reset Value** : 必须为16进制数, 以0x开头
11. **Access** : 默认配置支持RW/RO/RC/W1P/WO/WC, INT.STATUS, INT.MASK, INT.CLEAR (GROUP.REG_NAME 这种access名字表示这个寄存器在Reg Group内, Register/Group name列为Group的名字, Reg_name={Group_name}_{access.REG_NAME}) 支持编写cfg文件扩展类型。
12. 支持寄存器中的多个域类型不相同

4.3. itf格式

work_mode	module	hierarchy		汇总
MASTER	gpu	top.u_gpu		
mode	direction	type	name	description
signal	out	logic [31:0]	addr	address
struct	out	task_type	data	data
interface	NA	svt_axi_if	axi_mst	master axi interface
work_mode	module	hierarchy		汇总
SLAVE	cpu	top.u_gpu.u_cpu		
type	prefix	suffix	attr	description
AXI	axi_s_		addr_width=128 data_width=32 id_width=16 osd_num=8	slave axi interface
work_mode	module	hierarchy		汇总
MASTER	cpu	top.u_gpu.u_cpu		
type	prefix	suffix	attr	description
AHB	ahb_m_		addr_width=128 data_width=32	master ahb interface
work_mode	module	hierarchy		汇总
SLAVE	cpu	top.u_gpu.u_cpu		
type	prefix	suffix	attr	description
APB	apb_s_		addr_width=128 data_width=32	slave apb interface

1. "汇总"页签包含所有接口列表，点击接口链接可以跳转到相应接口页签
2. 页签名为接口名字，接口名尽量短，减少agent名字长度
3. work_mode：为接口的工作模式，可以填：MASTER/SLAVE/MONITOR
4. module：为接口所在RTL模块的名字
5. hierarchy：为接口在RTL中的例化层次，从top/harness开始
6. 汇总：点击汇总链接可以跳转到"汇总"页签，
7. type：端口类型，可为：signal/interface/标准接口(AXI/AHB/APB,ACE)
8. type=signal，分别填direction/type/name/description
9. type=interface，分别填type/name/description，direction可以填NA
10. type=标准接口，分别填prefix/suffix/attr/description
11. type=标准接口，prefix:填相对标准名字多出的前，suffix:填相对标准名字多出的后缀，attr(data/addr/id):填接口的配置信息
12. type=标准接口，目前支持AXI/ACE/APB/AHB,
13. type=signal和type=itf可以在一个表中，type=标准接口，必须独立为一个表，且一个表只有一个接口

5. genReg

生成UVM寄存器模型和寄存器RTL代码

5.0 介绍

1. excel表格支持寄存器数组，一个表格支持多个页签(对应一个reg_model和一个RTL的寄存器slave)
2. 生成寄存器模型，包含后门，包含覆盖率模型，支持寄存器数组，支持自定义寄存器field
3. 生成C程序头文件，包含所有寄存器的宏定义(数组寄存器提供指针)给软件编程使用
4. 生成寄存器RTL代码，支持APB/AHB/AXI-lite接口，支持用户定义的任意寄存器类型(支持SV模式和verilog模式)
5. 生成UVM寄存器测试用例

5.1 命令

```
stk genReg reg.xlsx -t data/reg -g -v
```

- t: 设置reg模型文件目录,不填写的情况下会使用默认的模式, 包含ro/wo/rw/w1c/w1p这几种类型的寄存器模型, 指定模型后, 可以覆盖默认模型。
- g: 表示excel文件格式带reg_group
- v: 表示使用verlog语句, 端口不使用数组
- log: 输出解析excel的信息

5.2 使用

- 前提: 已经有编辑好的寄存器表单excel文件, excel的格式可以参考stk.genExcel, 或者直接调用`stk genExcel --mode reg xxx.xlsx`生成寄存器表单的模板。
- 命令: `stk genReg xxx.xlsx --common common.json`
- 输出: 生成文件在./out目录下
 - reg_models/xxx_reg_model.sv-----> excel有多个表单页签, 就会有多个这种文件, 为页签中所以寄存器组成的reg_model
 - reg_models/xxx_reg_test.sv-----> 寄存器扫描测试用例, 需要修改地址
 - reg_models/xxx_reg_top_model.sv---> 这是所以xxx_reg_model的汇总
 - reg_models/svk_reg_test.sv-----> 寄存器扫描组件
 - reg_rtl/xxx_reg_macro.h-----> C程序使用的宏定义代码
 - reg_rtl/xxx_regs.v-----> 生成的REG RTL代码

common.json需要包含如下内容

- author: 用户名, 用于显示在生成文件的文件头中
- email: 邮箱地址, 用于显示在生成文件的文件头中
- proj: 项目名, 用于reg_top_mode, reg_pkg文件的命名

```
{
  "author": "Jakod",
  "email": "JakodYuan@outlook.com",
  "proj" : "pangu"
}
```

	A	B	C	D	E	F	G
1	Module Name	Owner	Data Width	Address Width	Base Address	Bus Type	
2	demo		32	64	0x0	APB	
3	Register Name	Address Offset	Field Bits	Field Name	Reset Value	Access	Field Discription
4	TEST0	0x80000000	17:2	pll	0x0123	RW	pll配置
5			1:1	soft_reset	0x0	RW	软复位:0:复位, 1:解复位
6			0	clk_gate	0x0	RW	时钟关断, 0:开启时钟, 1:关闭时钟
7	TEST1	0x80000010	1:1	data1_1	0x0	RO	值
8			0:0	data1_0	0x0	RO	值
9	TEST2	0x80000018	1:1	data2_1	0x0	RC	值
10			0:0	data2_0	0x0	RC	值
11	TEST3	0x80000020	0:0	data3	0x0	WC	值
12	TEST4	0x80000028	0:0	data4	0x0	W1C	值
13	TEST5	0x80000030	31:0	bit_mask	0x0	BMW	数据
14	TEST6	0x80000038	0:0	data6	0x0	W1P	值
15	ARRAY	0x80000040*10	15:0	coeff0	0xffff	CYC_CNT_RC	系数
16			31:16	coeff1	0xffff	CYC_CNT_RC	系数
17	TEST7	0x800000A0	0:0	data7	0x0	SAT_CYC_WC	值
18	TEST8	0x800000A4	31:0	bit_mask1	0x0	BMW	数据
19	SRC	0x800000A8	31:0	src	0x0	RO	
20	RAW	0x800000AC	31:0	raw	0x0	RO	
21	CLEAR	0x800000B0	31:0	clear	0x0	W1P	
22	MASK	0x800000B4	31:0	mask	0x0	RW	
23	STATUS	0x800000B8	31:0	status	0x0	RO	
24	ADDR_MSB	0x800000C0*10*0x8	31:0	addr_msb	0x0	RW	
25	ADDR_LSB	0x800000C4*10*0x8	31:0	addr_lsb	0x0	RW	
26	NOT_SAME	0x80000110	31:16	fro	0x0	RO	
27			15:8	frw	0x0	RW	
28			7:0	fwo	0x0	WO	
29							

5.3 寄存器模型

特性：

- 支持uvm_reg_field自定义，比如invert寄存器，使用默认的uvm_reg_field，则无法通过mirror与RTL比对通过。这里的解决方案分为3步：
 - 自定义一个invert_reg_field类，并把该文件添加到filelist中(在生成的寄存器模型之前)

```
class invert_reg_field extends uvm_reg_field;
    `uvm_object_utils(inv_reg_field)

    local static bit m_invert = define_access("INVERT"); // 定义一种访问类型

    function new(string name = "inv_reg_field");
        super.new(name);
    endfunction

    task post_write(uvm_reg_item rw);
        if(!predict(~rw.value[0]))
            `uvm_error("post_write", "predict is error")
        endtask
    endclass
```

- 将寄存器域的access列设置为INVERT，genReg例化的时候会例化invert_reg_field来代替，

Field Bits	Field Name	Reset Value	Access	Secure	Field Discription
0:0	test	0x0	INVERT	none	测试寄存器, 写1, 读取0, 写0, 读取1

access类型是RW/RO/RC/WC/WO/这些UVM支持的访问类型，会例化uvm_reg_field，否则会选例化access_reg_field代替uvm_reg_field待开发

2. 支持backdoor，在例化寄存器模型时，指定reg_bank的顶层路径，即可通过后门访问
3. 打包了一个reg_test用例，用户值需要修改地址即可进行寄存器扫描测试。
4. 生成一个通用的寄存器覆盖率模型
5. 生成C程序的头文件，包含所有寄存器的宏定义(数组寄存器提供指针)给软件编程使用。

5.4 RTL代码

生成的RTL代码可以选择使用APB、AHB、AXI-LITE接口，根据寄存器表单的bus type列选择。

- 寄存器例化名为寄存器地址(u_0x80000000)
- 寄存器类型module由用户提供，比如：RW_REG，RO_REG，`module_name=reg_type+"_REG"`
- 寄存器的端口和例化信息，根据用户提供的寄存器RTL模型进行例化连接，寄存器RTL模型，在后面介绍。

```
module DEMO_CFG;
    input                                pvalid;
    input [ADDR_WIDTH-1:0] paddr;
    input [DATA_WIDTH-1:0] pdata;
    input                                psel;
    input                                pwrite;
    input                                penable;
    output [DATA_WIDTH-1:0] prdata;
    output                                perror;
    output                                pready;

    output [16-1:0] pll;
    output [1-1:0] soft_reset;
    output [1-1:0] clk_gate;

    output [16-1:0] coff0_0;
    output [16-1:0] coff1_0;
    output [16-1:0] coff2_0;
    output [16-1:0] coff3_0;
    ....
    // TEST1
    wire TEST1_sel;
    wire [18-1:0] TEST1_out;
    wire [16-1:0] pll;
    wire [1-1:0] soft_reset;
    wire [1-1:0] clk_gate;
    assign TEST1_sel = paddr == 0x80000058 ? 1'b1:1'b0
    RW_REG u_0x80000000#(
        .WIDTH(18)
    )(
        .clk      (clk),
        .rst_n    (rst_n),
        .sel      (TEST1_sel),
        .in       (pdata),
        .out      (TEST1_out),
    );
    assign pll = TEST1_out[17:2];
```

```

assign soft_reset = TEST1_out[1:1];
assign clk_gate = TEST1_out[0:0];

always@(posedge clk)begin
    if(rst_n == 0)
        prdata <= 0;
    else begin
        case(paddr)
            0x80000000: prdata <= {pll,soft_reset,clk_gate}
            0x80000008: prdata <= {coff0_0,coff1_0,coff2_0,coff3_0}
            0x80000010: prdata <= {coff0_1,coff1_1,coff2_1,coff3_1}
            0x80000018: prdata <= {coff0_2,coff1_2,coff2_2,coff3_2}
            0x80000020: prdata <= {coff0_3,coff1_3,coff2_3,coff3_3}
            0x80000028: prdata <= {coff0_4,coff1_4,coff2_4,coff3_4}
            0x80000030: prdata <= {coff0_5,coff1_5,coff2_5,coff3_5}
            0x80000038: prdata <= {coff0_6,coff1_6,coff2_6,coff3_6}
            0x80000040: prdata <= {coff0_7,coff1_7,coff2_7,coff3_7}
            0x80000048: prdata <= {coff0_8,coff1_8,coff2_8,coff3_8}
            0x80000050: prdata <= {coff0_9,coff1_9,coff2_9,coff3_9}
            0x80000058: prdata <= {pll,soft_reset,clk_gate}
            default:prdata <= 0;
        endcase
    end
endmodule

```

- reg_group: 用于实现寄存器数组和寄存器之间的端口连接，group_type：NORMAL, INT
- reg
- reg_field
- 寄存器例化：
 - 例化名，REG_TYPE U_{{ADDR}}
 - 端口名，.field_name

5.5 寄存器RTL模型编写

寄存器的RTL模型通过yaml文件的形式呈现，根名字为类型名，每个类型5个方面的说明：

```

RW:
  TOP:
    - out FW {field}{arr}
  WIRE:
    - RW ? {reg}_wdata{arr}
    - 1 {reg}_wen{arr}
    - RW ? {reg}_out{arr}
  REG:
    - clk -> clk
    - rstn -> rstn
    - wen -> {reg}_wen{arr}
    - wdata -> {reg}_wdata{arr} ? extract(wdata)

```

```

- out    -> {reg}_out{arr}    ? {field}{arr}
CONN:
- extract ? {reg}_wdata{arr} = wdata
- assign    {reg}_wen{arr}    = (wen && waddr == {addr})
- split     ? {field}{arr}    = {reg}_out{arr}
REN: True
UVM: RW
RTL: RW_REG

```

5.1. TOP

TOP定义寄存器在reg_bank的端口，格式为：- dir width name

- dir：端口发现，可以为：in/out,
- width：端口位宽，可以为：num/RW/FW,
 - num，表示这个端口的位宽就是固定值
 - RW，端口位宽等于寄存器位宽，(约束：端口的name必须包含{reg})
 - FW，端口位宽等于寄存器域位宽，(约束：端口的name必须包含{field})
- name：为端口名，可以包含{reg}, {field}, {arr}等占位符，表示寄存器名、域名、数组宽度。(约束：name必须包含{reg}或者{field}，末尾必须包含{arr})
 - {reg}，会替换为寄存器名
 - {field}，会替换为field名，且产生的端口数量与field数量相同
 - {arr}，如果寄存器为寄存器数组，则会替换为数组宽度，否则为空

5.2. WIRE

WIRE定义寄存器例化是在reg_bank内部需要产生的wire信号，格式为：- width [?] name

- width：wire的位宽，可以为：num/RW/FW,
 - num，表示wire的位宽就是固定值
 - RW，wire位宽等于寄存器位宽，(约束：端口的name必须包含{reg})
 - FW，wire位宽等于寄存器域位宽，(约束：端口的name必须包含{field})
- ?: 可选标识，表示如果寄存器只有一个域，这个wire不需要添加，要与REG/CONN的可选表示配合使用(wire用了可选表示，在REG/CONN中使用wire的项也需要添加可选标识),比如：{reg}_wdata{arr}使用了可选标识，在REG的wdata端口和CONN的{reg}_wdata{arr} = wdata也要用可选标识
- name: 为wire名，可以包含{reg}, {field}, {arr}等占位符，表示寄存器名、域名、数组宽度。(约束：name必须包含{reg}或者{field}，末尾必须包含{arr})
 - {reg}，会替换为寄存器名
 - {field}，会替换为field名，且产生的端口数量与field数量相同
 - {arr}，如果寄存器为寄存器数组，则会替换为数组宽度，否则为空

5.3. REG

REG定义寄存器module的端口，格式为：- port_name -> conn_name ? conn_name2

- port_name: 寄存器端口名，不能有{reg},{field},{arr}这些占位符
- conn_name：实例化寄存器时，连接的信号名，这些信号名必须在TOP或者WIRE中定义过。除了clk,rstn,wdata这3个信号。
- ?: 可选标识符，表示寄存器只有多个域的时候，使用conn_name连接端口，否则用conn_name2连接端口

- `conn_name2`: 与`conn_name`相似，在寄存器只有一个域的时候使用。对信号可以使用`extract()`操作，表示在信号的末尾添加域range范围，比如`extract(wdata)=>wdata[f.msb:f.lsb]`

5.4. CONN

CONN定义寄存器相关信号的连接关系，使用的信号必须在TOP或者WIRE中定义过，除了`clk,rstn,wdata,wen,ren,waddr,raddr`这7个信号。格式为：`mode [?] left = right`

- `mode`: 连接模式，可以为：`assign/extract/split/merge`
 - `assign`，表示直接连接，最能转为1个`assign`语句，比如：`assign {reg}_wen{arr} = (wen && waddr == {addr})`
 - `extract`，`left`的位宽为`RW`，`right`的位宽为`data_width`，`extract`模式会把`right`中域对应range的信号提取出来，连接到`left`上，比如：`extract {reg}_wdata{arr} = wdata`等效于，`assign {reg}_wdata{arr}[pos+f.width:pos] = wdata[f.msb:f.lsb]`
 - `split`，`left`为`field`的信号，`right`为`reg`的信号，`split`模式会把`reg`的信号拆分成多个`field`的信号，比如：`split {field}{arr} = {reg}_out{arr}`等效于，`assign {field}{arr} = {reg}_out{arr}[pos+f.width:pos]`
 - `merge`，`left`为`reg`的信号，`right`为`field`的信号，`merge`模式会把`field`的信号合并成`reg`的信号，比如：`merge {reg}_out{arr} = {field}{arr}`等效于，`assign {reg}_out{arr}[pos+f.width:pos] = {field}{arr}`
- `?:` 可选标识符，表示寄存器只有多个域的时候，CONN的这项才会添加
- `left`: 连接的左侧信号，可以包含`{reg}`，`{field}`，`{arr}`等占位符，只能为一个信号
- `right`: 连接的右侧信号，可以包含`{reg}`，`{field}`，`{arr}`等占位符，`extract,split,merge`模式下，`right`只能为1个信号。`assign`模式下，`right`可以为多个信号，信号之间可以使逻辑操作符，比如：`{reg}_wdata{arr} = wdata & 0x3f`，还可以使用特殊的常量，比如`{RW}/{FW}/{addr}`，分别表示寄存器的位宽、域位宽、寄存器地址。

5.5. REN

`REN(Bool)`，表示寄存器是否可以读，可读的话，寄存器域根据地址译码，连接到`rdata`上。

5.6. UVM

寄存器在UVM寄存器模型中的类型，如果类型名字在UVM支持的类型中，会例化`uvm_reg_field`，如果不在支持的类型中，会例化`{model.UVM}_reg_field`，比如`INVERT.UVM: INVERT`，会例化`INVERT_reg_field`

5.7. RTL

设置例化寄存器的module名

`{field}{arr}`有特殊功能，即在连接`rdata`的信号固定为这个信号，`rdata = {{field1}{arr},{field0}{arr}}`，所以必须把连接`rdata`的信号定义为`{field}{arr}`

6. instRTL

生成verilog文件中module的例化，或者vhdl文件中entity的例化。生成例化的格式都是verilog格式的。

6.1 使用方式

- verilog

```
stk instRTL xxx.sv          # 生成的inst会打印在窗口中
stk instRTL xxx.sv -c      # 生成的inst拷贝到了剪切板中，可以直接在使用的地方粘贴即可
stk instRTL xxx.sv -f xx.txt # 生成的inst写入到文件中
```

- vhdl

```
stk instRTL -v xxx.v          # 生成的inst会打印在窗口中
stk instRTL -v xxx.v -c      # 生成的inst拷贝到了剪切板中，可以直接在使用的地方粘贴即可
stk instRTL -v xxx.v -f xx.txt # 生成的inst写入到文件中
```

6.2 RTL格式说明

无论verilog和还是vhdl，端口格式都很多样，如果考虑所以可能出现的格式，程序编写非常复杂。然而生僻的写法，基本上不会被使用，甚至有些公司规范不允许使用，所以，程序对输入的RTL端口格式不是完全支持，而是支持常用格式。

6.2.1 verilog

- ANSI写法

```
module module_name#(
    parameter name = default_value,          // 支持省略类型，default type is int
    parameter real name = default_value
)(
    input name,                               // 支持端口省略方向，省略类型
    output logic [7:0] name,
    logic [3:0] name,
    name,                                     // 支持类型方向同时省略
)
    parameter name = default_value;          // 支持在module block内添加parameter
endmodule
```

- Non-ANSI写法

```
module module_name(name0, name1, name2, name3)
    parameter name = default_value,          // 支持省略类型，default type is int
    parameter real name = default_value
    input name0;                             // 支持省略类型
    output logic [7:0] name1;
    output logic [3:0] name2, name3;          // 支持同时写多个端口信号
endmodule
```

- import
module u_xx后面包含import xx;语句

```

module module_name
    import xxxx::*;
##( 6.    parameter width = 1
)(
    ports
);

```

- include

port端口通过include文件的方式添加进来

```

module module_name(
    `include "rtl_port.sv"
);

```

6.2.2 vhdl

```

library ieee;
use ieee.std_logic_1164.all;

entity my_module is
    generic (
        NUM      : natural := 10;
        WIDTH    : natural := 8  // 参数WIDTH，默认值为8
    );
    port (
        data      : in  std_logic_vector(WIDTH-1 downto 0);
        clk,reset : in  std_logic;
信号          q      : out std_logic_vector(WIDTH-1 downto 0)
    );
end entity my_module;

```

6.3 不支持场景

7. parseLog

7.1 使用说明

parseLog用来解析log文件，得出编译和仿真PASS/FAIL的结论，解析根据3类关键词句来处理：

- m/must_contain:
log必须匹配该关键词句才能判定为PASS
区分大小写
- c/can_contain :
log的某一行匹配上该关键词句不会判定为FAIL，尽管该行匹配上了not_contain。该关键词句用来排除not_contain中可以包含的关键词句，比如not_contain=Error，但是log中出现'is not an Error'也会认定为错

误，设置`can_contain='is not an Error'`则可解决该问题

有一种特殊情况，一行中有多个错误信息，一个可以匹配`can_contain`一个不能匹配`con_contain`，依然会判定为FAIL。比如`UVM_ERROR is not an Error`，这里`UVM_ERROR`不在`can_contain`中，所以会判定为FAIL。

区分大小写

- `n/not_contain`:

`log`中某一行出现该关键词句会判断为FAIL，除非该行又匹配到了`can_contain`关键词句

不区分大小写

注意：

1. 多个关键词句通过','分隔，如果关键词句中包含','用','来代替
2. 如果关键词中有空格或者(,)等特殊字符，那么整个参数需要用单引号"或者双引号""来包裹，推荐无论关键词句是否有空格，整个参数值都用引号包裹`parseLog --can_contain 'is not an Error'`
3. 关键词句中包含了特殊字符，比如：`+`、`*`、`(`、`)`、`[`、`]`等正则表达式的特殊符合，需要对其添加"进行转义

7.2 特性

1. 逆向解析文件，提高解析效率，因为`log`如果报错都会出现在末尾，逆向解析可以提前得出解析结果，结束解析。
2. FAIL时，输出匹配到的错误信息，或者输出需要匹配而未匹配的关键词句
3. PASS时，输出匹配到需要匹配的信息

输出信息

- PASS

```
... // 匹配到的must_contain信息
PASS
```

- FAIL

```
... // 匹配的not_contain信息，或者提示某个must_contain的关键词句未被匹配到
FAIL
```

8. regress

在芯片验证过程中，会编写随机用例，通过大量回归来找到设计中潜在的BUG。用例回归其实是一个很简单的问题，就是让切换种子，让用例不断仿真，并对仿真出错的用例进行分析。

这个回归工具，根据配置支持自动切换种子，然后并行执行用例，回归的信息状态信息 (pass, fail, finish) 可以在`konsle`中实时显示，回归用例的具体信息 (比如哪条用例出错了，错误原因是什么，种子号是多少) 可以通过输出的`log`文件，或者使用`stk.regrServer`在网页中查看，推荐使用网页查看。

回归工具，读取`yaml`文件得到编译和仿真的参数，编辑简单，可读性也非常高。用例执行支持LSF提交，也支持本地的多进程执行。

8.1 功能

1. 支持一个命令完成编译和仿真
2. 支持多个build

3. 每个用例支持独立控制仿真次数
4. 支持随机种子随机，也可以指定种子
5. 回归结束后，对于错误的用例会生成一个rerun.yaml，可以基于这个yaml对错误用例进回归
6. 支持回归级别的prev_cmd和post_cmd，进行回归前的预处理和后处理
7. 支持用例级别的prev_cmd和post_cmd，进行用例前的预处理和后处理
8. 支持用例的fail_cmd和pass_cmd，分别在用例fail和pass时执行
9. 支持用例结束后进行log检查，log检查基于stk.parseLog
10. 支持并行度控制
11. 支持timeout控制
12. 支持本地模式和lsf模式，支持bsub参数控制
13. 支持跳过错误过多的用例
14. 支持结果存储到数据，通过网页显示
15. 支持3中用例执行分发模式：RR、WRR、SWRR，用例可以配置weight值来控制分发，仿真时间长的可以设置weight值大
16. yaml支持include、继承、引用等功能
17. 支持bsub提交错误后retry(没有log)
18. 输出return.yaml，可以对仿真错误的用例使用相同的选项和种子进行重新回归。

8.2 简单使用

1. 在执行回归前，必须先有一个yaml格式的回归配置文件，默认为当前目录下的regr.yaml文件，格式可以参考[配置文件](#)
2. 在命令行中执行stk工具 `stk regress --cfg xx.yaml`

```
##### SIMULATION #####
[PASSED]: one_apc_one_queue_one_auto_compute (123456)
[PASSED]: one_apc_one_queue_multi_auto_compute (one_apc_one_queue_multi_auto_compute)
[PASSED]: all_apc_one_queue_one_auto_compute (all_apc_one_queue_one_auto_compute)
[PASSED]: one_apc_one_queue_one_manual_compute (one_apc_one_queue_one_manual_compute)
[PASSED]: one_apc_one_queue_multi_manual_compute (one_apc_one_queue_multi_manual_compute)
[PASSED]: all_apc_one_queue_one_manual_compute (all_apc_one_queue_one_manual_compute)
[PASSED]: one_apc_one_queue_one_compute (one_apc_one_queue_one_compute)
[PASSED]: one_apc_one_queue_multi_compute (one_apc_one_queue_multi_compute)
[PASSED]: all_apc_one_queue_one_compute (all_apc_one_queue_one_compute)
```

3. 查看回归结果
 - 通过网页查看，这个可以参考stk.regrServer
 - 通过log文件查看，回归会在当前目录产生一个{regr_name}_{datetime}.log的文件。
 - 通过job_id(使用lsf是是bsub id，使用多进程则是pid),可以查询任务的具体状态。
4. rerun：使用输出的rerun.yaml文件，执行 `stk regress --cfg rerun.yaml`

8.3 配置文件

回归的配置的和要执行的任务信息通过一个yaml格式的配置文件承载，该文件能很简洁直观的方式编写和阅读。为了提高yaml的复用性和扩展性，在基本的yaml语法功能基础上，新增了：include，继承，引用节点，引用环境变量等功能，具体参考[yaml语法扩展](#)

8.3.1 文件结构

配置文件包含3个根节点 · regr_cfg、builds和simulations



```
my_lsf_cfg:
  queue: short
  mem: 1000

my_parse_cfg:
  must_contain: SIMULATION PASSED
  can_contain:
    - error\s\
    - lerror
    - \+error,UVM_ERROR
  not_contain: error,fail

regr_cfg:
  proj_name: Pangu
  subsys_name: GPU
  database_en: True
  timeout: 10
  dist_algo: WRR # RR/WRR/SWRR
  max_run_num: 1
  fail_skip_num: 0
  show_all_job_cmd: True
  retry_times: 1

prev_cmd:
  run_mode: local
  cmd_str: echo "Previous command"
  post_cmd: bsub -q short urg -full64 -dir $$ENV_ROOT_DIR/regress/work/cov/simv.vdb
    -dir $$ENV_ROOT_DIR/regress/work/mreged/ -dbname
  $$ENV_ROOT_DIR/regress/work/merged/new/test
    -parallel -maxjobs 10 -warn none -tgl portsonly

glb_cfg:
  run_mode: local
  lsf_cfg: $my_lsf_cfg
  parse_cfg: $my_parse_cfg

build_common:
  must_contain: ''

sim_common:
  seed: '{{SEED}}'
```

```

dir: work
rand_sim(sim_common):
  dir: $dir
  tc: ''
  pname: ''
  opt: ''
  times: 20
  sim_cmd: stk bsim -tc=$tc -seed=$seed -d=$dir $opt -p
$$ENV_ROOT_DIR/tc/test_cfg/$pname.cfg -pname $pname
  main_cmd: $sim_cmd -ccov
  fail_cmd:
    - rm -rf
$$ENV_ROOT_DIR/sim/$dir/cov/simv.vdb/snaps/coverage/db/testdata/$(tc)_$(pname)_$(seed)
d)
  - $sim_cmd -fsdb
  pass_cmd: rm -rf $log
  log: $dir/log/$(tc)_$(pname)_$(seed).log

test_sim(sim_common):
  dir: $dir
  tc: ''
  opt: ''
  times: 10
  sim_cmd: stk bsim -d=$dir -tc=$tc -seed=$seed -times=$times $opt -ccov
  main_cmd: $sim_cmd -ccov
  fail_cmd:
    - rm -rf
$$ENV_ROOT_DIR/sim/$dir/cov/simv.vdb/snaps/coverage/db/testdata/$(tc)_$(seed)
  - $sim_cmd -fsdb
  pass_cmd: rm -rf $log
  log: $dir/log/$(tc)_$(seed).log

builds:
  build0(build_common):
    work_dir: $dir
    main_cmd: stk bsim elab -d=$work_dir
    fail_cmd:
      - echo "Build failed"
      - echo "exit 1"
    log:
      - $work_dir/log/vh1_an.log
      - $work_dir/log/tb2_an.log
      - $work_dir/log/db3_an.log
      - $work_dir/log/elab.log
  build1(build_common):
    work_dir: debug
    main_cmd:
    log:
      - $work_dir/log/vh1_an.log
      - $work_dir/log/tb2_an.log
      - $work_dir/log/db3_an.log
      - $work_dir/log/elab.log

simulations:
  tc001_sanity(rand_sim):

```

```

    tc: tc001_sanity
    pname: random_cfg
    weight: 2

tc002_sanity(test_sim):
    tc: tc002_sanity
    weight: 3
    seeds:
[12345678,53534456,91565564,87894564,45656221,79156545,791523151,91345516,23454948,7913456]

```

- `regr_cfg`：为回归的相关配置信息，同时包含了`prev_cmd`和`post_cmd`，用于在`builds`和`simulations`前后执行。
- `builds`：用于放置编译的相关配置
- `simulations`：用于放置仿真相关配置

当然，配置文件为yaml格式的文件，并不约束添加其他的根节点，如果有需要可以新增额外的根节点，比如上面的：`my_lsf_cfg`，`my_parse_cfg`，`build_common`，`sim_common`，`rand_sim`，`test_sim`，通过定义`commom`节点，然后使用继承可以大量减少文本编辑，可读性也可以增强。

8.3.2 regr_cfg

`regr_cfg`必须包含如下对象，对象的类型值参考[yaml语法扩展]

对象名	值类型	说明
<code>proj_name</code>	<code>str</code>	项目名称
<code>subsys_name</code>	<code>str</code>	子系统名称
<code>database_en</code>	<code>bool(optional)</code>	只能填 <code>False</code> 和 <code>True</code> ，表示是否使用数据库，默认为 <code>True</code>
<code>timeout</code>	<code>float</code>	表示 <code>job</code> 超时时间，单位为分钟
<code>max_run_num</code>	<code>int</code>	并行执行用例数量，采用本地运行模式时，并行度不能太高，否则会因为进程太多，导致系统死机
<code>prev_cmd</code>	<code>cmd_format(optional)</code>	在执行回归开始前执行的命令，格式参考后续的 <code>cmd_format</code>
<code>post_cmd</code>	<code>cmd_format(optional)</code>	在执行回归结束后执行的命令，格式参考后续的 <code>cmd_format</code>
<code>glb_cfg</code>	<code>dict</code>	包含 <code>lsf_cfg/run_mode/parse_cfg</code>
<code>glb_cfg.lsf_cfg</code>	<code>str</code>	设置 <code>cmd</code> 使用 <code>bsub</code> 提交时的选项,比如 <code>bsub -q short -R "rusage[mem=1000]</code>
<code>glb_cfg.run_mode</code>	<code>enum</code>	设置 <code>cmd</code> 的默认运行模式： <code>local/lsf</code>
<code>glb_cfg.parse_cfg</code>	<code>dict</code>	配置解析命令的参数，包含 <code>must_contain/can_contain/not_contain</code>

对象名	值类型	说明
glb_cfg.parse_cfg.must_contain	list[str]/str	log解析参数，log中必须包含的关键字，多个用","分隔，没有关键词句可以写空/null/None，具体说明参考regress.parseLog
glb_cfg.parse_cfg.not_contain	list[str]/str	log解析参数，log中不能包含的关键字，多个用","分隔，没有关键词句可以写空/null/None，具体说明参考regress.parseLog
glb_cfg.parse_cfg.can_contain	list[str]/str	log解析参数，log中可以包含的关键字，多个用","分隔，没有关键词句可以写空/null/None，具体说明参考regress.parseLog
fail_skip_num	int(optional)	用例错误多少条以后，会跳过这条用例剩余的测试,默认为0，表示出错后不跳过错误用例
show_all_job_cmd	bool(optional)	False: 默认只在网页或者log中显示main job的命令，True：显示所有job的命令，perv_cmd/mina_cmd/post_cmd/pass_cmd/fail_cmd, 默认值为False
retry_times	int(optional)	在main_cmd执行后，没有log输出时，会重试main_job，默认值为0，不重试
dist_algo	str(optional)	用例之间的分发算法，包括：RR/WRR/SWRR，需要与用例的weight配置一起使用，默认为RR
max_process	int(optional)	local执行是并行提交的进程数，默认为10
lsf_update_interval	int(optional)	lsf job更新时间间隔
lsf_kill_interval	int(optional)	lsf job kill时间间隔
db_flush_interval	int(optional)	数据库下刷时间间隔
lsf_mode	enum(optional)	cli/api, 默认为cli

8.3.3 builds

builds中可以包含多个job_group，每个job_group对象必须包含如下对象

对象名	值类型	说明
prev_cmd	cmd_format(optional)	build前执行的命令
main_cmd	cmd_format	build执行的命令
post_cmd	cmd_format(optional)	build后执行的命令
pass_cmd	cmd_format(optional)	build成功后执行的命令
fail_cmd	cmd_format(optional)	build失败后执行的命令

对象名	值类型	说明
log	list[str]/str(optional)	log文件的路径和名字，路径为一个相对当前目录的路径，可以有多个，工具会自动根据log文件的个数和must/can/not_contain参数，产生对应数量的log解析命令，添加到main_cmd列表的末尾。如果不想不进行log解析，可以将log的值填成None或者Null
must_contain	list[str]/str(optional)	同regr_cfg.glb_cfg.parse_cfg.must_contain, 默认使用regr_cfg.glb_cfg.parse_cfg.must_contain
can_contain	list[str]/str(optional)	同regr_cfg.glb_cfg.parse_cfg.can_contain, 默认使用regr_cfg.glb_cfg.parse_cfg.can_contain
not_contain	list[str]/str(optional)	同regr_cfg.glb_cfg.parse_cfg.not_contain, 默认使用regr_cfg.glb_cfg.parse_cfg.not_contain

8.3.4 simulations

simulations中可以包含多个job_group，每个job_group对象必须包含如下对象

对象名	值类型	说明
prev_cmd	cmd_format(optional)	同build
main_cmd	cmd_format	同build
post_cmd	cmd_format(optional)	同build
pass_cmd	cmd_format(optional)	同build
fail_cmd	cmd_format(optional)	同build
log	list[str]/str(optional)	同build
must_contain	list[str]/str(optional)	同regr_cfg.glb_cfg.parse_cfg.must_contain, 默认使用regr_cfg.glb_cfg.parse_cfg.must_contain
can_contain	list[str]/str(optional)	同regr_cfg.glb_cfg.parse_cfg.can_contain, 默认使用regr_cfg.glb_cfg.parse_cfg.can_contain
not_contain	list[str]/str(optional)	同regr_cfg.glb_cfg.parse_cfg.not_contain, 默认使用regr_cfg.glb_cfg.parse_cfg.not_contain
times	int	job_group的执行次数
weight	int(optional)	(optional)job_group的权重
seeds	list(optional)	(optional)设置随机种子，用例回归时依次取seeds中的值替换{{SEED}}，times>seeds.size(),后续的{{SEED}}进行随机

8.3.5 cmd_format

cmd_format可以包含一条命令，也可以包含多条命令，每条命令可以用简略模式和详细模式两种方式编写

实际执行的命令由3部分控制：cmd_str, run_mode, lsf_cfg

- run_mode=local，任务在本地机器执行

```
cmd = cmd_str
```

- run_mode=lsf，任务通过bsub提交给LSF进行执行

```
cmd = lsf_cfg + cmd_str
```

8.3.5.1 单一命令

- 简略模式

```
main_cmd: stk bsim -tc=$tc -seed=$seed
```

等效于

```
main_cmd:
  cmd_str: stk bsim -tc=$tc -seed=$seed
  run_mode: $regr_cfg.glb_cfg.run_mode
  lsf_cfg: $regr_cfg.glb_cfg.lsf_cfg
```

- 详细模式

该模式用于命令不使用glb_cfg的配置进行提交的情况

```
main_cmd:
  cmd_str: stk bsim -tc=$tc -seed=$seed
  run_mode: lsf
  lsf_cfg: bsub -q long -R 'rusage[mem=100]'
```

8.3.5.2 多条命令

- 简略模式

```
main_cmd:
- stk bsim -a -i 1 # analysis
- stk bsim -a -i 2 # analysis
- stk bsim -b      # elaboration
```

等效于

```
main_cmd:
-
```

```

cmd_str: stk bsim -a -i 1 # analysis
run_mode: $regr_cfg.glb_cfg.run_mode
lsf_cfg: $regr_cfg.glb_cfg.lsf_cfg
-
cmd_str: stk bsim -a -i 2 # analysis
run_mode: $regr_cfg.glb_cfg.run_mode
lsf_cfg: $regr_cfg.glb_cfg.lsf_cfg
-
cmd_str: stk bsim -b # elaboration
run_mode: $regr_cfg.glb_cfg.run_mode
lsf_cfg: $regr_cfg.glb_cfg.lsf_cfg

```

- 详细模式

```

main_cmd:
-
  cmd_str: stk bsim -a -i 1 # analysis
  run_mode: local # 本地模式可以不配置lsf_cfg
-
  cmd_str: stk bsim -a -i 2 # analysis
  run_mode: lsf
  lsf_cfg: bsub -q long -R 'rusage[mem=100]'
-
  cmd_str: stk bsim -b # elaboration
  run_mode: lsf
  lsf_cfg: bsub -q long -R 'rusage[mem=100]'

```

- 混合模式

```

main_cmd:
-
  cmd_str: stk bsim -a -i 1 # analysis
  run_mode: local
- stk bsim -a -i 2 # analysis
- stk bsim -b # elaboration

```

8.3.5.3 长命令

在命令很长的时候，写在一行不太好看，由2种方式写成多行

1. 基于yaml的特性

```

cmd_str: stk bsim -tc=$tc
        -seed=$seed

```

2. 把命令拆分为list

```
cmd_str:
- stk bsim -tc=$tc
- -seed=$seed
```

列表合并时会添加一个空格

上面两种方式等效于

```
cmd_str: stk bsim -tc=$tc -seed=$seed
```

8.3.6 global cfg

regr_cfg下有glb_cfg配置，包含run_mode,lsf_cfg,parse_cfg。run_mode和lsf_cfg会成为没有配置run_mode和lsf_cfg命令的默认配置。parse_cfg会成为build/simulation task的默认配置。

- run_mode
设置所有命令的默认执行模式：local,lsf
- lsf_cfg
设置所有命令的默认bsub选项，比如**bsub -q short -R 'rusage[mem=10]'**
- parse_cfg
配置解析build/simulation task生成log的过滤的关键字，每个关键字通过","分隔，也可以list方式编写，包含：
 - must_contain：必须包含的关键字
 - not_contain：不能包含的关键字
 - can_contain：可以包含的关键字
解析通过的条件是log中包含must_contain内容的行，不包含匹配了not_contain但是没有匹配can_contain的行。

```
parse_cfg:
must_contain: PASSED
can_contain:
- error(s),lerror
- +error,UVM_ERROR,maxfail,not contain error\,fail
not_contain: error,fail
```

如何关键字中包含","，那么","需要进行转义，即"\,"，上面的**not contain error\,fail**，实际的关键字为**not contain error,fail**

8.3.6.1 使用glb_cfg配置

1. cmd_format中不显示设置run_mode和lsf_cfg，那么cmd会从glb_cfg中获取run_mode和lsf_cfg

```
main_cmd: stk bsim -tc=$tc -seed=$seed
```

这条命令没有配置run_mode和lsf_cfg, 那么run_mode和lsf_cfg会从glb_cfg中获取, 等效于

```
main_cmd:
  cmd_str: stk bsim -tc=$tc -seed=$seed
  run_mode: $regr_cfg.glb_cfg.run_mode
  lsf_cfg: $regr_cfg.glb_cfg.lsf_cfg
```

2. build/simulation task中显示设置must_contain/can_contain/not_contain, 那么build/simulation task会使用glb_cfg.parse_cfg中配置的must_contain/can_contain/not_contain

```
builds:
  build0:
    main_cmd: stk bsim -tc=$tc -seed=$seed
```

等效于

```
builds:
  build0:
    main_cmd: stk bsim -tc=$tc -seed=$seed -b
    must_contain: $regr_cfg.glb_cfg.parse_cfg.must_contain
    can_contain: $regr_cfg.glb_cfg.parse_cfg.can_contain
    not_contain: $regr_cfg.glb_cfg.parse_cfg.not_contain
```

8.3.6.2 覆盖glb_cfg

1. cmd_format中显示设置run_mode和lsf_cfg, 那么cmd会使用配置的run_mode和lsf_cfg

```
main_cmd:
  cmd_str: stk bsim -tc=$tc -seed=$seed
  run_mode: lsf
  lsf_cfg: bsub -q short -R 'rusage[mem=100]'
```

2. build/simulation task中显示设置parse_cfg, 那么parse_cfg会覆盖glb_cfg中的parse_cfg

```
builds:
  build0:
    main_cmd: stk bsim -tc=$tc -seed=$seed -b
    must_contain: # 没有必须包含的关键字
    can_contain: 'error(s),lerror'
    not_contain: error,fail
```

8.3.6.3 分发配置

1. max_run_num: 并行运行的任务数量
2. timeout: 任务超时后会被kill
3. dist_algo: 分发算法，可选值：RR/WRR/SWRR

```
simulations:
  sim1:
    times: 5
    weight: 2
  sim2:
    times: 5
    weight: 1
  sim3:
    times: 5
    weight: 3
```

- RR: 每个simulation task一次分发

```
sim1, sim2, sim3, | sim1, sim2, sim3 | sim1, sim2, sim3 | sim1, sim2, sim3 | sim1,
sim2, sim3
```

- WRR: 每个simulation task按照simulation task的weight进行分发

```
sim1, sim1, sim2, sim3, sim3, sim3 | sim1, sim1, sim2, sim3, sim3 | sim1, sim2 |
sim2 | sim2
```

- SWRR: 滑动权重方式分发

```
sim1, sim2, sim3, sim1, sim3, sim3 | sim1, sim2, sim3, sim1, sim3 | sim1, sim2 |
sim2 | sim2
```

推荐：仿真时间比较长的用例提供比较大的weight，让其尽早提交。

8.3.7 demo

bsim.yaml

```
bsim_cfg(bsim_base_cfg):
  com:
    flists:
      - vhd1|work|$(ENV_ROOT_DIR)/cfg/vh.f
      - vlog|tb|$(ENV_ROOT_DIR)/cfg/tb.f
      - vlog|abc|$(ENV_ROOT_DIR)/cfg/db.f
```

base_regr.yaml

```

regr_cfg:
  proj_name: Pangu
  subsys_name: GPU
  database_en: True
  timeout: 10
  dist_algo: WRR # RR/WRR/SWRR
  max_run_num: 1000
  retry_times: 1
  prev_cmd:
  post_cmd: bsub -q short urg -full64 -dir $$ENV_ROOT_DIR/regress/work/cov/simv.vdb
            -dir $$ENV_ROOT_DIR/regress/work/mreged/ -dbname
            $$ENV_ROOT_DIR/regress/work/merged/new/test
            -parallel -maxjobs 10 -warn none -tgl portsonly
  glb_cfg:
    run_mode: lsf
    lsf_cfg: bsub -q short -R "rusage[mem=16]"
    parse_cfg:
      must_contain: SIMULATION PASSED
      can_contain:
        - error\s\
        - lerror
        - \+error,UVM_ERROR
      not_contain: error,fail

build_common:
  must_contain: # 没有must_contain关键字

sim_common:
  seed: '{{SEED}}'
  weight: 1
  times: 1

## 8. 随机用例，通过配置cfg来实现用例
rand_sim(sim_common):
  work_dir: work
  tc: tc001_random
  opt: ''
  times: 10
  pname: pname
  run_cmd: stk bsim -tc=$tc -seed=$seed -work_dir=$work_dir -
p=$$(ENV_ROOT_DIR)/tc/cfg/$pname.cfg -pname=$pname $opt
  fail_cmd: stk bsim -tc=$tc -seed=$seed -work_dir=debug -
p=$$(ENV_ROOT_DIR)/tc/cfg/$pname.cfg -pname=$pname $opt -fsdb # 出错后dump波形，使用
的是debug的build进行仿真
  pass_cmd: rm -rf $(work_dir)/log/$(tc)_$(pname)_$(seed).log # pass后删除log
  log: $(work_dir)/log/$(tc)_$(pname)_$(seed).log

## 8. 直接用例
test_sim(sim_common):
  work_dir: work
  tc: tc001_sanity
  opt: ''
  times: 10
  run_cmd: stk bsim -tc=$tc -work_dir=$work_dir -seed=$seed $opt

```

```
fail_cmd: stk bsim -tc=$tc -work_dir=debug -seed=$seed $opt -fsdb
pass_cmd: rm -rf $(work_dir)/log/$(tc)_$(seed).log
log: $(work_dir)/log/$(tc)_$(seed).log
```

username_regr.yaml

```
builds:
  build0(build_common):
    work_dir: work
    run_cmd: stk bsim -work_dir=$work_dir -step2 -b -ns
    log: $work_dir/log/cmp.log

  build1(build_common):
    work_dir: debug
    run_cmd: stk bsim -work_dir=$work_dir -step3 -b -ns -fsdb
    log:
      - $work_dir/log/vh1_an.log
      - $work_dir/log/tb2_an.log
      - $work_dir/log/db3_an.log
      - $work_dir/log/elab.log

simulations:
  tc001_sanity(test_sim):
    tc: tc001_direct_test1

  tc002_sanity(test_sim):
    tc: tc002_direct_test2

  tc003_sanity(test_sim):
    tc: tc002_direct_test2
    opt: -so '+SVK_MAX_TIME=30000'

  random1(rand_sim):
    tc: tc001_sanity
    pname: pname1
    seeds: [132456, 468465, 56565] # 指定仿真的种子，如果times>seeds多，后面的会使用随机种子。
    times: 3

  random2(rand_sim):
    tc: tc003_sanity
    pname: pname2
    times: 0 # 这条用例不仿真
```

8.4 任务组成与提交

regress是一个任务提交和管理的工具，任务分为3个层级，task/job_group/job。

8.4.1 任务组成

- task : jobgroup的集合

- regr_cfg: 包含prev_cmd和post_cmd两个job_group
- build : 包含prev_cmd/main_cmd/post_cmd/fail_cmd/pass_cmd 5个job_group
- simulation : 同build
- job_group : job的集合 , job_group可以包含1个或者多个job
 - regr_cfg.prev_cmd <可选>
 - regr_cfg.post_cmd <可选>
 - build.prev_cmd <可选>
 - build.main_cmd
 - build.post_cmd <可选>
 - build.fail_cmd <可选>
 - build.pass_cmd <可选>
 - simulation.prev_cmd <可选>
 - simulation.main_cmd
 - simulation.post_cmd <可选>
 - simulation.fail_cmd <可选>
 - simulation.pass_cmd <可选>
- job : 就是一条要执行的命令

note : build和simulation的log配置不为空的时候 , build.post_cmd和simulation.post_cmd中会添加stk
 parseLog -m build.must_contain -n build.not_contain -c build.can_contain build.log命令 , log有多少个文件就会添加多少个命令
 注意 : main_cmd这个job_group的job数量必须多于或者等于log的文件数 , 且main_cmd[i] <-> log[i]进行对应

8.4.2 执行顺序

执行顺序的规则如下:

1. regr_cfg.prev_cmd -> builds.build_tasks -> simulations.sim_tasks -> post_cmd这4个单元是顺序执行的
2. builds和simulations下的task是并行执行的 , 并行度由配置regr_cfg.max_run_num控制。
3. task内的job_group是串行执行的 , 顺序为prev_cmd -> main_cmd -> post_cmd -> fail_cmd -> pass_cmd
4. job_group内部的多个job是串行执行的

fail_cmd和pass_cmd是根据前面job执行的状态选择执行的 , 如果前面的job都是pass的则执行pass_cmd, 否则执行fail_cmd

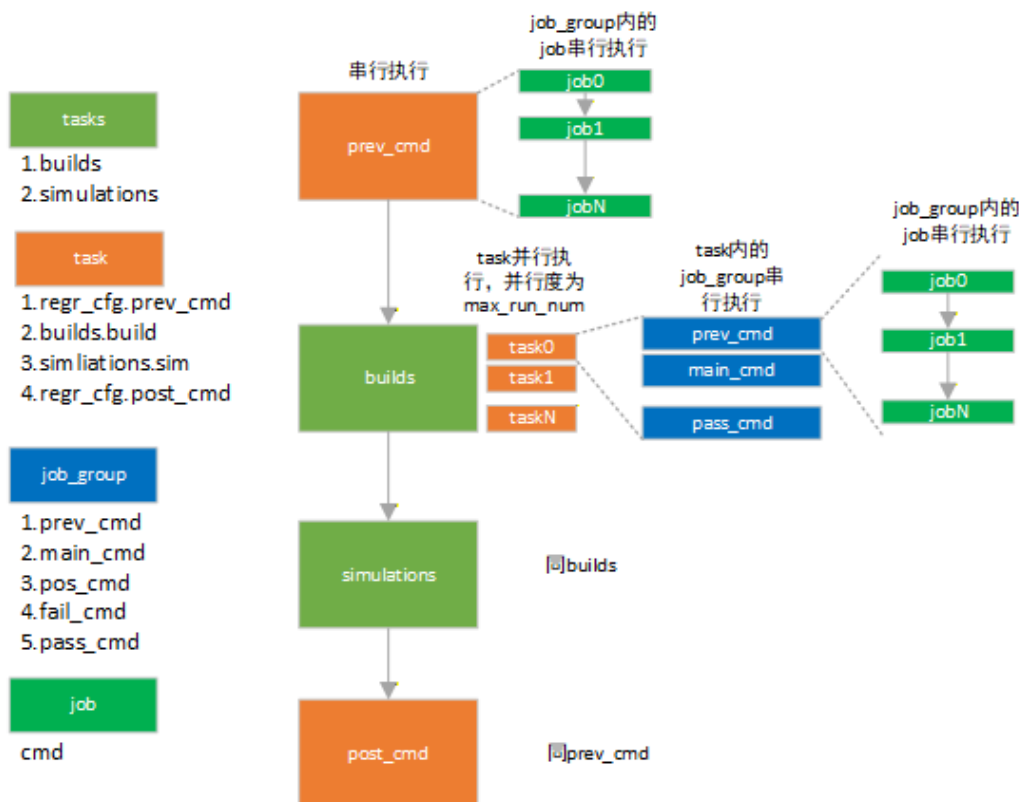
用sv的语法描述为如下 :

```
begin
  regr_cfg.prev_cmd;
  repeat(regr_cfg.max_run_num)begin
    fork
      builds.build_tasks;
    join_none
  end
  repeat(regr_cfg.max_run_num)begin
    fork
      simulations.sim_tasks;
    join_none
  end
end
```

```

end
regr_cfg.post_cmd;
end

```



1. 先串行执行prev_cmd这个job_group内部的所有job
2. 执行完prev_job，然后并行执行builds内部的task
3. builds中的所以task都执行完以后，执行simulations中的所有task
4. simulations中的task都执行完以后，执行post_cmd这个job_group

task的执行顺序，参考[dist_algo](#)

异常情况：

1. 在执行prev_cmd/builds/simulations/post_cmd期间，如果用户使用ctrl+c退出程序，程序会立刻结束所有任务，更新报告和数据库，然后退出整个回归。
2. 如过prev_cmd执行的任务有报错或异常退出的情况，回归直接结束
3. 执行builds中有job报错或者异常退出，会跳过simulations，直接进入post_cmd

8.4.3 结束状态

每个job执行完都会有一个状态(PASS/FAIL/KILL/TIMEOUT)，task根据其所有job的结束状态得到其结束状态
job状态：

- PASS：下面的4种情况都满足才为PASS
 1. job为busb命令，则提交正常
 2. job执行正常结束
 3. job执行没有超时
 4. 如果job为一条log解析命令，则log解析结果为PASS
- FAIL：程序非超时和kill导致的结束，满足下面一个条件则为FAIL

1. job为busb命令，提交可能异常
 2. job执行异常结束(比如编译报错，仿真报错等，程序结束返回非0值)
 3. 如果job为一条log解析命令，则log解析结果为FAIL
- TIMEOUT：程序正常执行，但是执行时间超过regr_cfg.timeout的时间
 - KILL：因为用户ctrl+c导致的job结束，如果该任务为bsub任务，则在退出前会bkill掉，如果是多进程任务，则会kill掉执行的进程再退出。

task状态：

- PASS：task下的所有job的状态都为PASS
- FAIL：task下有一个job为FAIL
- TIMEOUT：task下有一个job为TIMEOUT
- KILL：task下有一个job为KILL

因为task下的job是串行执行的，只有前一个job执行结果为PASS/FAIL才会继续执行下一个job，如果job结果为TIMEOUT/KILL那么这个job后面的job不继续执行，整个task结束。即task结束条件为，所以job都结束了，或者有一个job为TIMEOUT/KILL。

8.5 参数选项

```
options:
-h, --help          show this help message and exit
--cfg CFG           configure file for regress, default=regr.yaml
--report_job_cmd    default display logfile, add this option replace as cmd in report file
--show_job_info     show job start and finish status
--show_start_info   show job/jobgroup start info
--debug            debug mode
-nb                skip build before simulation
```

- cfg
即回归的配置yaml文件
- report_job_cmd
用于控制log文件的中显示内容，不添加该选项，默认显示log文件，添加则显示job的命令信息
- show_job_info
默认terminal只显示jog_group的结束状态信息，添加该选项则会显示job的结束状态信息
- show_start_info
默认terminal只显示job_group/job的结束状态信息，添加该信息则会显示job_group/job的开始执行的信息
- debug
测试模式使用，实际使用中禁止使用

9. regrServer

为了方便回归信息的查看，管理和统计，regrServer工具以网页表格，图表等形式，非常直观的展示回归数据，方便项目管理。

9.1 使用说明

9.1.1 参数说明与使用

参数选项	说明
-cfg	配置文件

参数选项	说明
-use_admin	是否使用隐藏的admin账号，该账号推荐只有第一次设管理员的时候使用
-dev	开发模式，不使用gunicorn
-start_db	启动服务器前启动数据库

配置文件格式

```
use_matplotlib: false
view_need_login: true
gunicorn:
  bind: 0.0.0.0:8888
  workers: 4
  timeout: 120
  worker_class: sync
  max_requests: 500
  accesslog: server.log
  pidfile: pid.log
```

- view_need_login: 查看回归数据是否需要登录，默认不登录也可以查看|
- use_matplotlib: 默认情况下使用javascript，然而如果浏览器比较老旧，特别是linux的firefox，则不支持该功能，此时必须添加use_matplotlib选项用matplotlib来显示图表
- gunicorn: gunicorn的配置项
 - bind: 设置网页端口
 - workers: 工作进程数
 - timeout: 超时设置
 - worker_class: 同步模式
 - max_requests: 单个worker处理请求的最大数量
 - accesslog: 日志文件
 - pidfile: pid.log

9.1.2 启动

1. 推荐使用命令 `stk regrServer --cfg config.yaml -start_db`
2. `ifconfig`查看本机IP
3. 在浏览器中输入：`ip:8888`
4. 当然如果觉得IP难记和输入，可以修改/etc/hosts,添加`127.18.133.6 stk`，同时刷新DNS或者重启网络，在浏览器输入`stk:8888`即可访问。

```
## 9. 刷新DNS
ipconfig /flushdns           # windows
sudo systemctl status NetworkManager  # 查看NetworkManager状态
sudo systemctl start NetworkManager   # 启动NM
sudo service networking restart      # 重启NM
```

9.1.3 数据页面说明

stk.regrServer将stk.regress回归产生的数据以网页的形式展现出来。网页分为5个层级。

层级	说明
index	列出所有项目的汇总信息，包括总回归数，通过数等，还可以点击graph，查看分时段的统计数据。
subsys	列出子系统的汇总信息，包括总回归数，通过数等，还可以点击graph，查看分时段的统计数据。
proj	列出某个项目下，所有用户的回归信息，也可以通过点击graph，查看每个用户的分时段统计数据
user	列出某个用户的所以回归的汇总信息，点击某个回归可以查看回归的具体信息
regr	列出回归的任务的具体信息，这些任务的排列顺序为TIMEOUT->FAIL->KILL->PASS，即错误用例会显示在最前面，不同状态标记为不同的颜色
case	列出回归的所有用例的统计信息
log	网页显示log内容，并对包含error和fail的行进行高亮，管理员设置好各个用户的display后，还可以点击链接后，通过gvim打开log文件

9.2 页面展示

每个页面左上侧有一个导航，右上侧有一个登录/退出链接，如果登录的账号权限为4，那么左上方还会显示后台链接。未登录时右上方显示login链接，登录成功后显示"Welcome, xxx!",点击会显示logout的下拉菜单。

9.2.1 index

SVK.Regress

JakodYuan © 2024 Copyright

index

Login

project	pass	fail	finish	run	idle	all	pass_ratio	graph
Kirin	9	23	32	0	0	32	28.12 %	graph
PanGu	10	22	32	0	0	32	31.25 %	graph
TaiJi	149	378	527	0	44	578	28.27 %	graph

登录的是账号权限为4的时候，表格的右侧有删除按钮，可以删除整个项目的数据。

SVK.Regress

JakodYuan © 2024 Copyright

index [admin]

Welcome, admin!

project	pass	fail	finish	run	idle	all	pass_ratio	graph	
Kirin	9	23	32	0	0	32	28.12 %	graph	Delete
PanGu	10	22	32	0	0	32	31.25 %	graph	Delete
TaiJi	149	378	527	0	44	578	28.27 %	graph	Delete

9.2.2 proj

登录账户账号的权限等于4时，在表格的末尾显示delete按钮，删除整个用户的数据

SVK.Regress

JakodYuan © 2024 Copyright

index > Taiji [admin]

Welcome, admin!

Id	User	pass	fail	finish	run	idle	all	pass_ratio	graph	
1	USER0	55	144	199	0	25	224	27.64 %	graph	logout Delete
2	USER1	30	74	104	0	19	128	28.85 %	graph	Delete
3	USER2	64	160	224	0	0	226	28.57 %	graph	Delete

9.2.3 subsys

index > Pangu [admin]

Welcome, admin!

Id	subsys	pass	fail	finish	run	skip	idle	all	pass_rate	graph	
1	GPU	21	0	21	0	0	0	21	100.00 %	show	Delete

9.2.4 user

登录账户名字与数据用户名相同，且权限=2时，或者账号的权限大于等于3时，在表格的末尾显示delete按钮，删除整个回归的数据

SVK.Regress

JakodYuan © 2024 Copyright

index > Taiji > USER2 [admin]

Welcome, Jakod!

id	regr_name	b/c/r	status	run_time	pass	fail	finish	run	idle	all	pass_ratio	
1	_2024-06-12_142702	2,2,5	FINISH	0:1:5	9	23	32	0	0	32	28.12 %	Delete
2	_2024-06-07_174321	2,2,5	FINISH	0:1:10	10	22	32	0	0	32	31.25 %	Delete
3	_2024-06-07_173522	2,2,5	FINISH	0:1:10	8	24	32	0	0	32	25.00 %	Delete
4	_2024-06-07_170503	2,2,5	FINISH	0:1:10	9	23	32	0	0	32	28.12 %	Delete
5	_2024-06-07_165918	2,2,5	FINISH	0:0:55	8	24	32	0	0	32	25.00 %	Delete
6	_2024-06-07_165053	2,2,5	RUN	0:0:0	0	0	0	0	0	2	0.00 %	Delete
7	_2024-06-07_143259	2,2,5	FINISH	0:1:20	10	22	32	0	0	32	31.25 %	Delete
8	_2024-06-07_111753	2,2,5	FINISH	0:1:20	10	22	32	0	0	32	31.25 %	Delete

- regr_name：回归名=yaml.regr_cfg.regr_name+datetime
- build/case/p：分别显示:总jobgroup数(build数量，testcase数量，执行并行度)
- status：回归的状态，分别有：BUILD/RUN/FINISH
- run_time：回归的时间
- pass：分别在build/run阶段显示pass的jobgroup数量
- fail：同上
- finish：同上
- run：同上
- idle：同上
- all：同上

- pass_ratio: 同上



SVK.Regress

JakodYuan © 2024 Copyright

index > Kirin > USER0

Login

id	regr_name	build/case/p	status	run_time	pass	fail	finish	run	idle	all	pass_ratio
1	pds_2024-06-25_175551	4(2/2/5)	FINISH	0:0:28	6	24	30	0	0	30	20.00 %
2	pds_2024-06-21_154017	4(2/2/5)	RUN	0:0:0	0	0	0	5	25	30	0.00 %
3	pds_2024-06-20_183754	4(2/2/5)	RUN	0:0:0	0	0	0	5	25	30	0.00 %
4	pds_2024-06-19_105959	4(2/2/5)	FINISH	0:0:26	26	4	30	0	0	30	86.67 %
5	pds_2024-06-19_105837	4(2/2/5)	RUN	0:0:0	0	0	0	5	25	30	0.00 %
6	pds_2024-06-19_105700	4(2/2/5)	BUILD	0:0:0	0	0	0	2	0	2	0.00 %
7	pds_2024-06-19_105437	4(2/2/5)	FINISH	0:0:8	1	5	6	0	24	30	16.67 %
8	pds_2024-06-18_183818	4(2/2/5)	BUILD	0:00:00	0	0	0	0	2	2	0.00 %
9	pds_2024-06-18_175906	4(2/2/5)	FINISH	0:0:2	0	2	2	0	0	2	0.00 %
10	pds_2024-06-17_163341	4(2/2/5)	FINISH	0:0:19	5	27	32	0	0	32	15.62 %

«

1

2

3

»

9.2.4 regr

点击user页面的"regr_name"页面会调整到regr页面，该显示每天用例的状态、命令，log等相关信息，排列顺序为TIMEOUT->FAIL->KILL->PASS,且用不同颜色表示。

SVK.Regress

JakodYuan © 2024 Copyright

index > Taiji > USER1 > pds_2024-06-12_141633 [admin]

Welcome, admin!

id	job_name	job_id	log	cmd	status	info	run_time
21	tc001_sanity_17	46 -1	base_fun/log/tc001_sanity_479384.log	bsub -R "rusage[mem=1]" make ncrun mode=base_fun ccov=off wave=null seed=479384 tc=tc001_sanity bsub -R "rusage[mem=1]" svk parseLog base_fun/log/tc001_sanity_479384.log --can_contain "error\(\s\).error\(+error,UVM_ERROR" --not_contain error,fail	FAILED	submit bsub job is Failed, 'bsub' 不是内部或外部命令，也不是可运行的程序或批处理文件。	0:0:5
22	tc001_sanity_18	49 54	base_fun/log/tc001_sanity_372902.log	bsub -R "rusage[mem=1]" make ncrun mode=base_fun ccov=off wave=null seed=372902 tc=tc001_sanity bsub -R "rusage[mem=1]" svk parseLog base_fun/log/tc001_sanity_372902.log --can_contain "error\(\s\).error\(+error,UVM_ERROR" --not_contain error,fail	FAILED	base_fun/log/tc001_sanity_279809.log is not exists!	0:0:10
23	tc001_sanity_19	48 54	base_fun/log/tc001_sanity_218535.log	bsub -R "rusage[mem=1]" make ncrun mode=base_fun ccov=off wave=null seed=218535 tc=tc001_sanity bsub -R "rusage[mem=1]" svk parseLog base_fun/log/tc001_sanity_218535.log --can_contain "error\(\s\).error\(+error,UVM_ERROR" --not_contain error,fail	FAILED	base_fun/log/tc001_sanity_631312.log is not exists!	0:0:10
24	tc002_adm_sanity_0	76		bsub -R "rusage[mem=1]" make ncrun mode=debug ccov=off wave=null seed=363402 tc=tc002_adm_sanity	PASSED		0:0:5
25	tc002_adm_sanity_2	82		bsub -R "rusage[mem=1]" make ncrun mode=debug ccov=off wave=null seed=369345 tc=tc002_adm_sanity	PASSED		0:0:5
26	tc002_adm_sanity_3	49		bsub -R "rusage[mem=1]" make ncrun mode=debug ccov=off wave=null seed=198128 tc=tc002_adm_sanity	PASSED		0:0:5
27	tc002_adm_sanity_4	96		bsub -R "rusage[mem=1]" make ncrun mode=debug ccov=off wave=null seed=135014 tc=tc002_adm_sanity	PASSED		0:0:5
28	tc002_adm_sanity_5	71		bsub -R "rusage[mem=1]" make ncrun mode=debug ccov=off wave=null seed=196343 tc=tc002_adm_sanity	PASSED		0:0:5
29	tc002_adm_sanity_6	84		bsub -R "rusage[mem=1]" make ncrun mode=debug ccov=off wave=null seed=118558 tc=tc002_adm_sanity	PASSED		0:0:5
30	tc002_adm_sanity_7	42		bsub -R "rusage[mem=1]" make ncrun mode=debug ccov=off wave=null seed=704446 tc=tc002_adm_sanity	PASSED		0:0:5
31	tc002_adm_sanity_8	96		bsub -R "rusage[mem=1]" make ncrun mode=debug ccov=off wave=null seed=973876 tc=tc002_adm_sanity	PASSED		0:0:5
32	tc002_adm_sanity_9	68		bsub -R "rusage[mem=1]" make ncrun mode=debug ccov=off wave=null seed=179496 tc=tc002_adm_sanity	PASSED		0:0:5

< 1 2 >

SVK.Regress

JakodYuan © 2024 Copyright

index > Kirin > USER2 > pds_2024-06-14_181652

Login

id	job_name	job_id	log	cmd	status	info	run_time
1	build0	55 -1 -1 -1 -1	base_fun/log/pds_vhdlan.log base_fun/log/pds_vlogan.log base_fun/log/elab.cmp.log	bsub -R "rusage[mem=1]" make anall mode=base_fun udc+=define+POST_SIM bsub -R "rusage[mem=1]" make elab mode=base_fun ccov=off wave=null partcomp=off step.mode=3step bsub -R "rusage[mem=1]" svk parseLog base_fun/log/pds_vhdlan.log --must_contain "" --can_contain "error\(\s\).error\(+error,UVM_ERROR" --not_contain error,fail bsub -R "rusage[mem=1]" svk parseLog base_fun/log/pds_vlogan.log --must_contain "" --can_contain "error\(\s\).error\(+error,UVM_ERROR" --not_contain error,fail bsub -R "rusage[mem=1]" svk parseLog base_fun/log/elab.cmp.log --must_contain "" --can_contain "error\(\s\).error\(+error,UVM_ERROR" --not_contain error,fail	TIMEOUT	LSF job 55 is timeout	0:0:10
2	build1	74 -1 -1 -1 -1	debug/log/pds_vhdlan.log debug/log/pds_vlogan.log debug/log/elab.cmp.log	bsub -R "rusage[mem=1]" make anall mode=debug udc+=define+POST_SIM bsub -R "rusage[mem=1]" make elab mode=debug ccov=off wave=null partcomp=off step.mode=3step bsub -R "rusage[mem=1]" svk parseLog debug/log/pds_vhdlan.log --must_contain "" --can_contain "error\(\s\).error\(+error,UVM_ERROR" --not_contain error,fail bsub -R "rusage[mem=1]" svk parseLog debug/log/pds_vlogan.log --must_contain "" --can_contain "error\(\s\).error\(+error,UVM_ERROR" --not_contain error,fail bsub -R "rusage[mem=1]" svk parseLog debug/log/elab.cmp.log --must_contain "" --can_contain "error\(\s\).error\(+error,UVM_ERROR" --not_contain error,fail	TIMEOUT	LSF job 74 is timeout	0:0:10
3	tc001_sanity_0	10 26	base_fun/log/tc001_sanity_191068.log	bsub -R "rusage[mem=1]" make ncrun mode=base_fun ccov=off wave=null seed=191068 tc=tc001_sanity bsub -R "rusage[mem=1]" svk parseLog base_fun/log/tc001_sanity_191068.log --can_contain "error\(\s\).error\(+error,UVM_ERROR" --not_contain error,fail	TIMEOUT	LSF job 26 is timeout	0:0:15
4	tc002_adm_sanity_2	54		bsub -R "rusage[mem=1]" make ncrun mode=debug ccov=off wave=null seed=656050 tc=tc002_adm_sanity	TIMEOUT	LSF job 54 is timeout	0:0:10
5	tc002_adm_sanity_0	62		bsub -R "rusage[mem=1]" make ncrun mode=debug ccov=off wave=null seed=803766 tc=tc002_adm_sanity	FAILED	submit bsub job is Failed, 'bsub' 不是内部或外部命令，也不是可运行的程序或批处理文件。	0:0:10
6	tc001_sanity_1	26 29	base_fun/log/tc001_sanity_756923.log	bsub -R "rusage[mem=1]" make ncrun mode=base_fun ccov=off wave=null seed=756923 tc=tc001_sanity bsub -R "rusage[mem=1]" svk parseLog base_fun/log/tc001_sanity_756923.log --can_contain "error\(\s\).error\(+error,UVM_ERROR" --not_contain error,fail	FAILED	submit bsub job is Failed, 'bsub' 不是内部或外部命令，也不是可运行的程序或批处理文件。	0:0:10
7	tc001_sanity_2	47 -1	base_fun/log/tc001_sanity_437867.log	bsub -R "rusage[mem=1]" make ncrun mode=base_fun ccov=off wave=null seed=437867 tc=tc001_sanity bsub -R "rusage[mem=1]" svk parseLog base_fun/log/tc001_sanity_437867.log --can_contain "error\(\s\).error\(+error,UVM_ERROR" --not_contain error,fail	FAILED	submit bsub job is Failed, 'bsub' 不是内部或外部命令，也不是可运行的程序或批处理文件。	0:0:20
8	tc001_sanity_3	25 -1	base_fun/log/tc001_sanity_376220.log	bsub -R "rusage[mem=1]" make ncrun mode=base_fun ccov=off wave=null seed=376220 tc=tc001_sanity bsub -R "rusage[mem=1]" svk parseLog base_fun/log/tc001_sanity_376220.log --can_contain "error\(\s\).error\(+error,UVM_ERROR" --not_contain error,fail	FAILED	submit bsub job is Failed, 'bsub' 不是内部或外部命令，也不是可运行的程序或批处理文件。	0:0:15
9	tc002_adm_sanity_3	94		bsub -R "rusage[mem=1]" make ncrun mode=debug ccov=off wave=null seed=333638 tc=tc002_adm_sanity	FAILED	submit bsub job is Failed, 'bsub' 不是内部或外部命令，也不是可运行的程序或批处理文件。	0:0:10
10	tc001_sanity_4	30 90	base_fun/log/tc001_sanity_532661.log	bsub -R "rusage[mem=1]" make ncrun mode=base_fun ccov=off wave=null seed=532661 tc=tc001_sanity bsub -R "rusage[mem=1]" svk parseLog base_fun/log/tc001_sanity_532661.log --can_contain "error\(\s\).error\(+error,UVM_ERROR" --not_contain error,fail	FAILED	base_fun/log/tc001_sanity_532661.log is not exists!	0:0:10
11	tc002_adm_sanity_5	55		bsub -R "rusage[mem=1]" make ncrun mode=debug ccov=off wave=null seed=722291 tc=tc002_adm_sanity	FAILED	submit bsub job is Failed, 'bsub' 不是内部或外部命令，也不是可运行的程序或批处理文件。	0:0:4
12	tc002_adm_sanity_4	81		bsub -R "rusage[mem=1]" make ncrun mode=debug ccov=off wave=null seed=951384 tc=tc002_adm_sanity bsub -R "rusage[mem=1]" make ncrun mode=base_fun ccov=off wave=null seed=479363 tc=tc001_sanity	KILLED	LSF job 81 is killed	0:0:4

13	tc001_sanity_5	53	-1	base_fun/log/tc001_sanity_479363.log	bsub -R "rusage[mem=1]" svk parseLog base_fun/log/tc001_sanity_479363.log --can_contain "error(s)\terror\+error.UVM_ERROR" --not_contain error.fail	KILLED	LSF job 53 is killed	0:0:5
14	tc001_sanity_6	-1	-1	base_fun/log/tc001_sanity_140733.log	bsub -R "rusage[mem=1]" make ncrun mode=base_fun ccov=off wave=null seed=140733 tc=tc001_sanity	KILLED	LSF job -1 is killed	0:0:0
15	tc002_adm_sanity_6	-1			bsub -R "rusage[mem=1]" svk parseLog base_fun/log/tc001_sanity_140733.log --can_contain "error(s)\terror\+error.UVM_ERROR" --not_contain error.fail	KILLED	LSF job -1 is killed	0:0:0
16	tc001_sanity_7	-1	-1	base_fun/log/tc001_sanity_265907.log	bsub -R "rusage[mem=1]" make ncrun mode=debug ccov=off wave=null seed=208101 tc=tc002_adm_sanity	KILLED	LSF job -1 is killed	0:0:0
17	tc002_adm_sanity_1	51			bsub -R "rusage[mem=1]" svk parseLog base_fun/log/tc001_sanity_265907.log --can_contain "error(s)\terror\+error.UVM_ERROR" --not_contain error.fail	KILLED	LSF job -1 is killed	0:0:0
					bsub -R "rusage[mem=1]" make ncrun mode=debug ccov=off wave=null seed=275056 tc=tc002_adm_sanity			0:0:5

如果用户登录了，且查看回归的user_name与登录的用户名相同，那么在页面顶部会有一个"open log with gvim"的单选框，选中后点击log链接会通过gvim的方式打开log文件(目的是gvim都个性化高亮了错误内容，且方便在log中跳转和查找)。如果没有登录或者没有选中单选框，点击log链接会跳转到log页面显示log。

9.2.5 testcase

点击user页面的"build/case/p"列，会跳转到testcase页面，该页面为build和testcase的列表，点击"+"会显示各个build/testcase的详细信息，再次点击"- "，会折叠该信息。

JakodYuan © 2024 Copyright

index > Kirin > USER0 > pds_2024-06-25_175551

open log with gvim: ☐

Welcome, USER0!

id	test_name	pass_num/all_num	pass_ratio	
1	+	build0	0/1	0.0%
2	+	build1	0/1	0.0%
3	+	tc001_sanity	0/20	0.0%
4	-	tc002_adm_sanity	7/10	0.7%

id	log_id	cmd	status	info	run_time
1	63	bsub -R "rusage[mem=1]" make ncrun mode=debug ccov=off wave=null seed=53629524 tc=tc002_adm_sanity	TIMEOUT	LSF job 63 is timeout	0:0:3
2	14	bsub -R "rusage[mem=1]" make ncrun mode=debug ccov=off wave=null seed=56691705 tc=tc002_adm_sanity	FAILED	submit bsub job is Failed, 'bsub' 不是内部或外部命令，也不是可运行的程序或批处理文件。	0:0:2
3	71	bsub -R "rusage[mem=1]" make ncrun mode=debug ccov=off wave=null seed=30997569 tc=tc002_adm_sanity	FAILED	submit bsub job is Failed, 'bsub' 不是内部或外部命令，也不是可运行的程序或批处理文件。	0:0:2
4	15	bsub -R "rusage[mem=1]" make ncrun mode=debug ccov=off wave=null seed=51898546 tc=tc002_adm_sanity	PASSED		0:0:1
5	57	bsub -R "rusage[mem=1]" make ncrun mode=debug ccov=off wave=null seed=50690100 tc=tc002_adm_sanity	PASSED		0:0:1

与regr页面一样，如果用户登录了，且查看回归的user_name与登录的用户名相同，那么在页面顶部会有一个"open log with gvim"的选项。

9.2.5 log

显示log内容，并对包含error和fail的行进行高亮

SVK.Regress

JakodYuan © 2024 Copyright

index > Kirin > USER0 > pds_2024-06-13_150555 > base_fun/log/pds_vlogan.log

Login

1 abc ada

2 UVM_ERROR is not an error,s

3 adsf

4 failed

5 TEST IS PASS

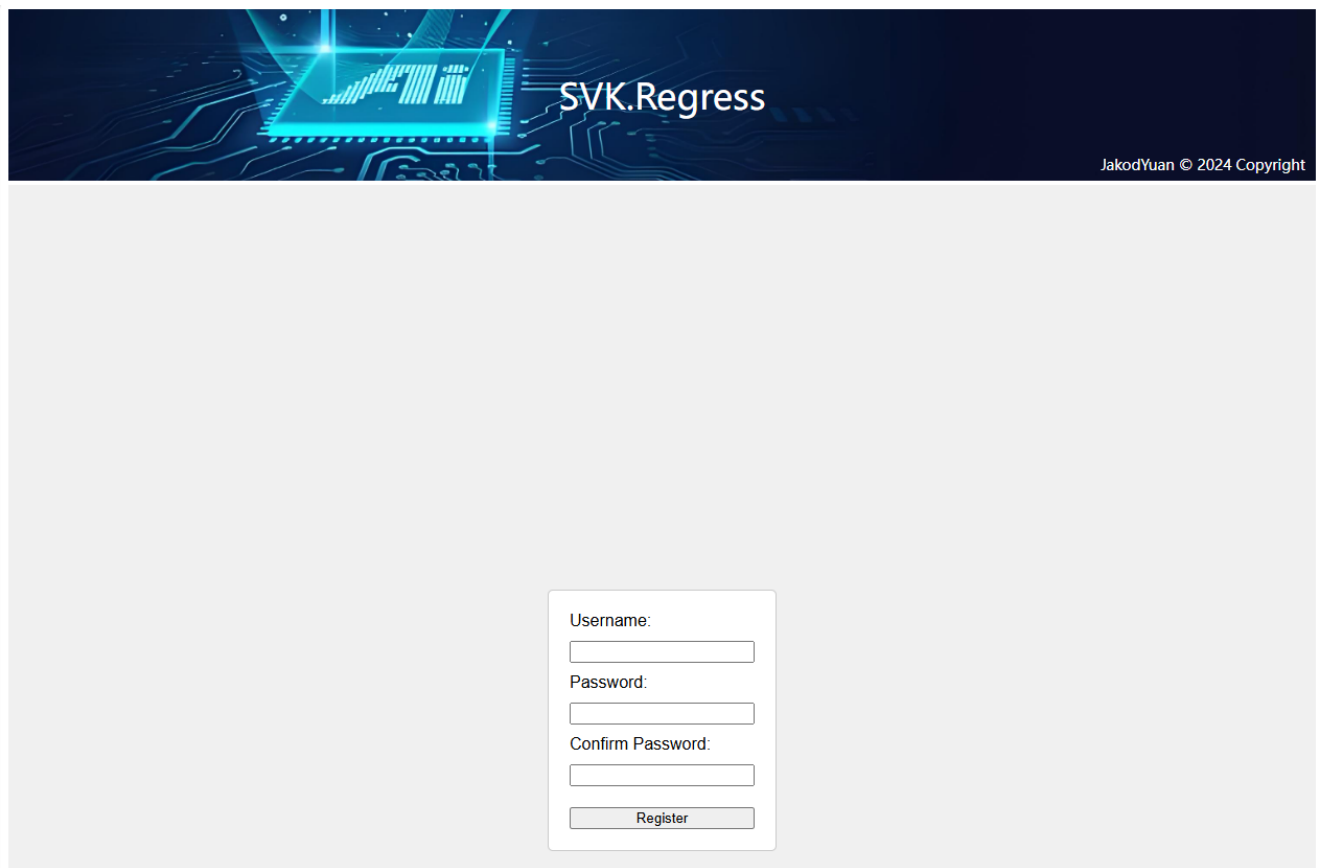
6 SIMULATION FINISHED

7

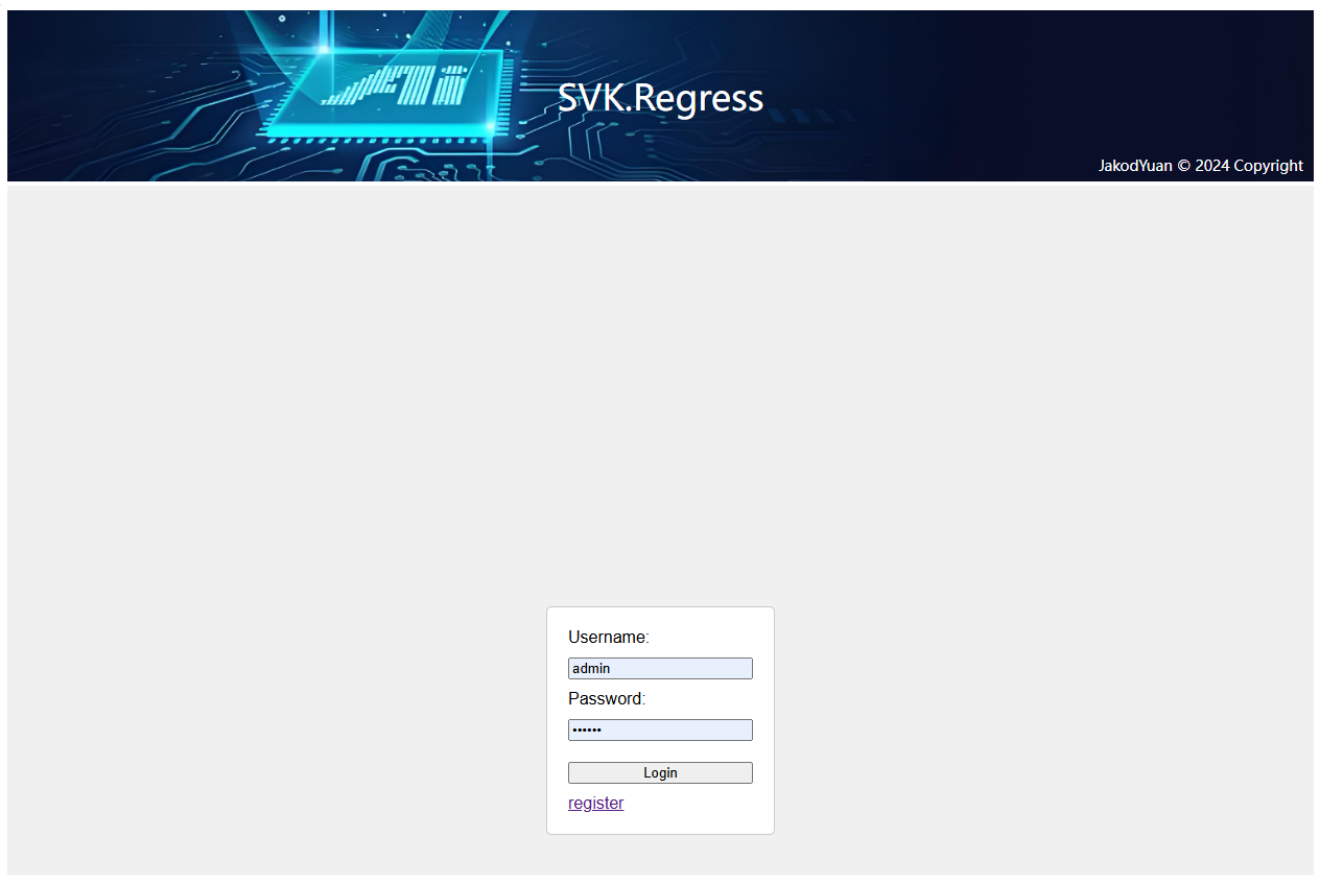
8

9 PASS

9.2.6 register

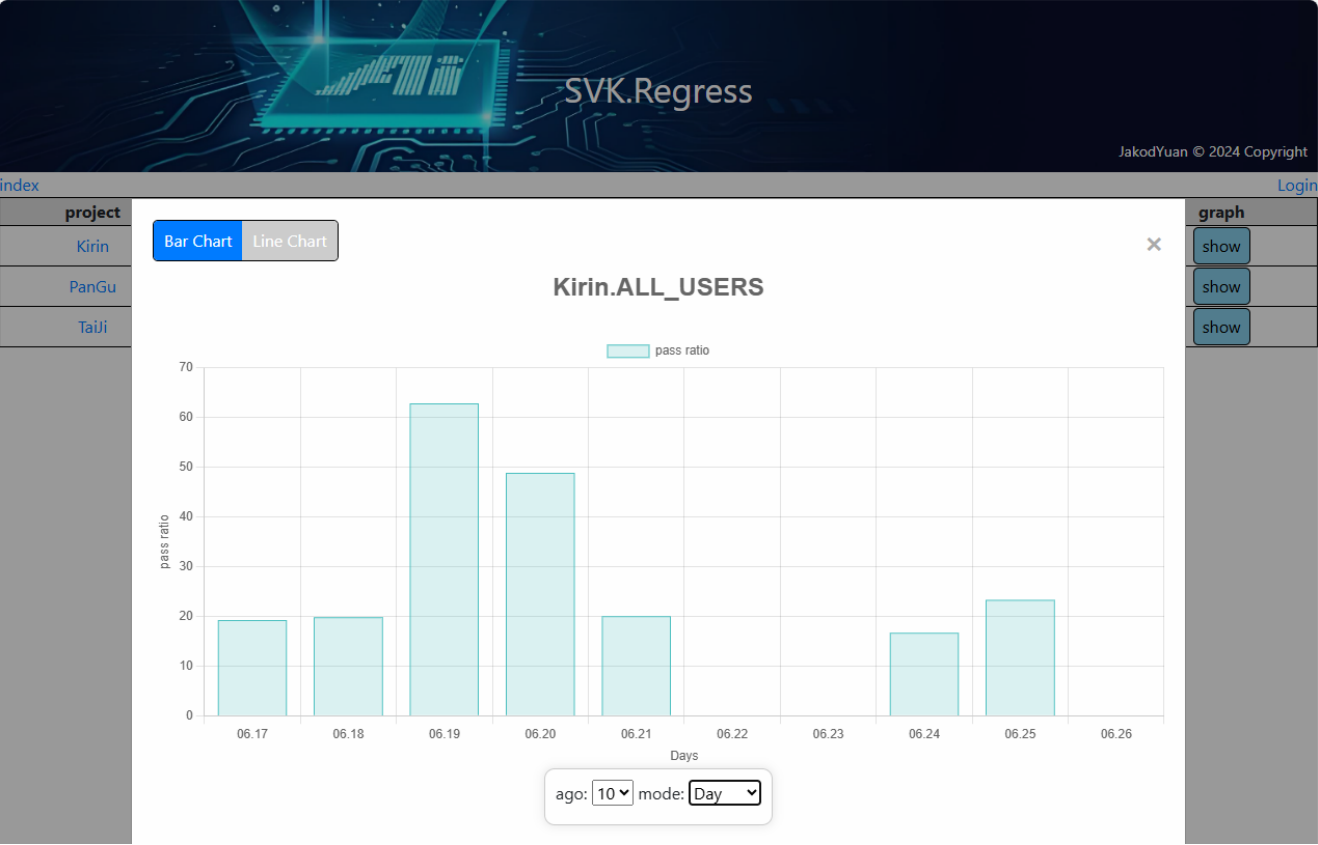


9.2.7 login

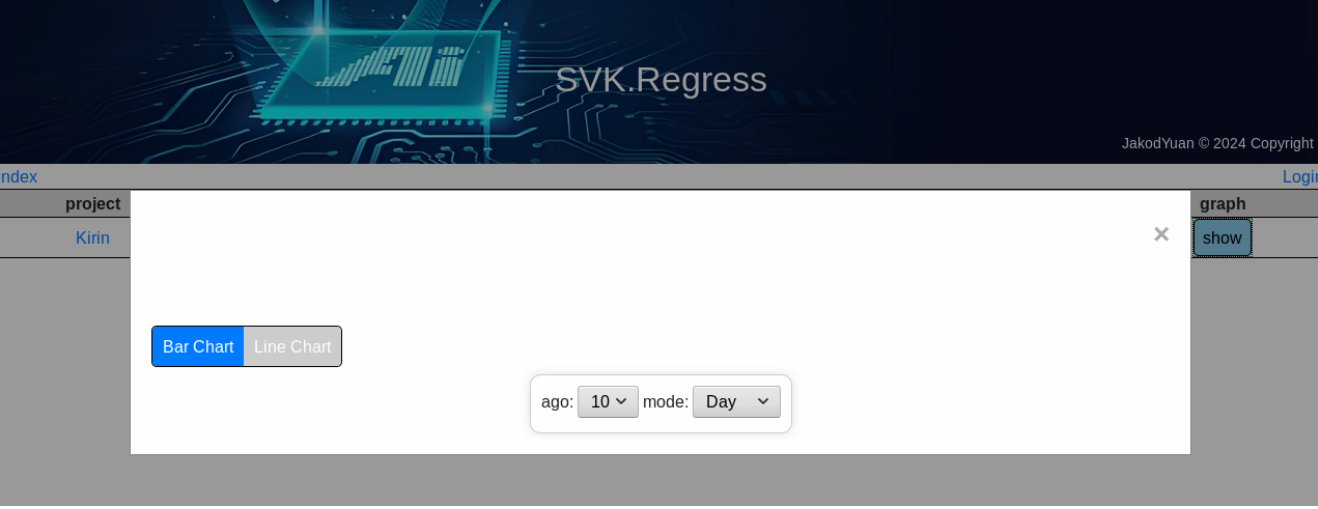


9.2.8 graph

如果regrServer没有使用-use_matplotlib选项且浏览器支持javascript的新特性，在index和proj页面点击show会悬浮显示图表



如果浏览器不支持javascript新特性，则为如下显示：



即无法显示图表，此时有两个选择：

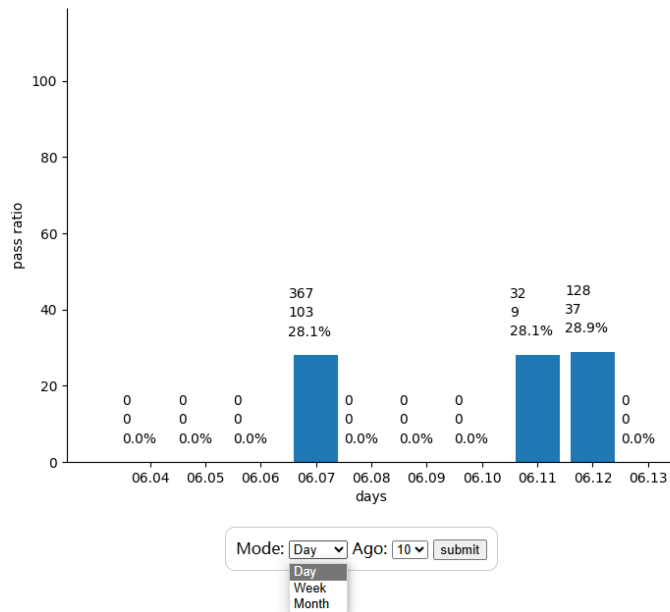
1. 启动服务器的时候使用`-use_matplotlib`选项，不使用javascript来显示图表，点击show则会显示如下页面



[index](#) [\[admin\]](#)

Welcome, Jakod!

Regress Bar Graph



2. 更新浏览器，以firefox为例，步骤如下：

1. 下载最新firefox:<https://www.mozilla.org/en-US/firefox/new/>
2. 解压：`tar xjf firefox-*.tar.bz2`
3. 移动解压文件到opt目录`sudo mv firefox /opt/`
4. 添加链接到bin目录下，以便可以直接调用firefox：`sudo ln -s /opt/firefox/firefox /usr/bin/firefox`
5. 启动新的firefox:`firefox`

10. genDummy

读取1个或者多个verilog源文件，生成对应的dummy.v文件。

读取一个verilog源文件，可以指定生成文件路径和名字。读取多个源文件是，生成文件名为源文件名加上"_dummy"后缀。

10.1 用法

```
stk genDummy <verilog_file> [-o output_file] [-d define] [-i inc_dir]
stk genDummy <verilog_file1> <verilog_file2>... [<output_file>] [-d define] [-i
inc_dir]
```

10. 实例

```
stk genDummy my_module.v -d MY_DEFINE -i ./include -o my_module_dummy.v
stk genDummy my_module1.v my_module2.v -d MY_DEFINE -i ./include
```

10.2 驱动说明

1. 对于module的input端口，不做处理
2. 对于module的输出端口，如果名字包含rdy/ready会驱动为1， 否则驱动为0.

11. stk.instConn

11.1 功能说明

instConn用于多个模块集成成一个顶层模块，且模块之间只有信号连接关系，没有逻辑处理。instConn可以根据配置文件中的信息，自动例化module，把有相同conn_name的端口通过wire连接起来，没有相同conn_name的端口则作为顶层端口。

1. 支持读取多个module文件，生成一个顶层文件。
2. module文件支持例化多次
3. port_name和conn_name支持通配符匹配
4. 支持module中包含\include和`define`宏
5. 端口支持结构体和interface
6. 端口支持一维数组
7. 支持添加额外组合逻辑,生成的信号不能是数组。
8. 信号位宽和维度检查

11.2 使用命令

```
stk instConn -cfg <cfg_file>
```

11.3 配置文件示例

1. files: 指定多个rtl源文件
2. inc_dirs: 指定搜索路径，用于处理include宏
3. defines: 指定宏定义,用于处理define宏
4. insts: 指定例化module的名称，端口连接关系，实例化次数
 - name: module名称 + module实例名称 + 实例化次数，实例化次数可以省略，省略时默认1次 (module名称必须在源文件中能找到)
 - conn: conn下面是一个列表，可以添加多个port_name和conn_name的映射关系，支持通配符匹配。且匹配是从前往后，如果前面匹配到了，后面就不再匹配。
 1. 如果port_name被某个conn[i].port_name匹配到了，那么conn_name为conn[i].conn_name

```
conn:
- axi_slv*    -> axi*    # axi_slv_awvalid会被axi_slv*匹配到，那么
conn_name为axi_awvalid
                        # .axi_slv_awvalid(axi_awvalid)
```

2. 如果模块例化多次，可以在conn_name中添加<i>占位符，<i>会被替换成实例化索引。<i>可以与数组配合使用[<i>].

```

conn:
  - data      -> data[<i>]  # .data(data[0]) or .data(data[1]) or
    .data(data[2])
  - data      -> data<i>    # .data(data0) or .data(data1) or .data(data2)

```

3. "用于匹配端口的一部分字符，并这部分字符替换conn_name中"位置，所以port_name和conn_name的'*'个数必须相同。
4. 如果port_name没有被任何conn[i].port_name匹配到，则默认被`-port_name -> port_name`匹配到，即conn_name为port_name。（如果module例化多次，需要添加一个conn_name包含<i>的默认匹配规则，比如`'* -> *_{i>}'`）
5. conn_name对应信号属性控制，`->`表示conn_name是wire还是port根据conn_name连接端口来决定，`=>`表示强制conn_name为端口，`~>`表示强制conn_name为wire。

5. top: 指定顶层模块名称

6. out_file: 指定输出文件名

7. logics: `logic [msb:lsb] signal_name = expression;`，指定额外组合逻辑，生成的信号不能是数组。生成信号必须指定位宽，即使是1bit信号

```

files:
  - rtl1.sv
  - rtl2.sv
  - rtl3.sv
inc_dirs:
  - './out'
  - './'

defines:
  - ABC1

insts:
  - name: rtl1 u_rtl1 * 2
    conn:
      - axi_slv*      -> axi*<i>
      - p*            -> apb_p*<i>
      - xxds          -> {1'b1, xxds<i>, 1'b0}
      - abc*          -> {abcx*<i>, 1'b0}
      - '*'           -> *_{i>'
  - name: rtl2 u_rtl2
    conn:
      - apb_*         => apb_*
      - axi_mst_*     -> axi_*
  - name: rtl3 u_rtl3
    conn:
      - axi_mst_*     -> axi_*
top: top
out_file: top.sv
logics:
  - logic [0:0] yyds = axi_awaddr0 == 12'h123;

```

11.4 wire和port规则

1. 如果在port_name与conn_name的关系中设置强制port或者wire，那么对于conn_name会强制成端口或者wire

```
conn:
- axi_slv*      => axi*<i> # 使用=>强制端口
- p*           ~> apb_p*<i> # 使用~>强制wire
```

2. 在没有强制的情况下，logic和struct类型的端口，如果conn_name中连接了1+个input端口，1个output端口，则会生成一个wire信号。
3. conn_name只连接1+个input端口，没有连接output端口，则会生成一个input端口。
4. conn_name只连接一个output端口，没有连接input端口，则会生成一个output端口。
5. 如果conn_name连接多个output端口，则报错，多驱动。

额外添加的logic信号也遵守该规则，添加logic信号可认为是某个例化模块的output端口

11.5 parameter规则

每个例化模块的parameter会连接到顶层模块同名parameter端口上,如果多module有相同的parameter，会连接同一个parameter端口, top模块parameter的默认值为第一个module的parameter默认值。

```
module top #(
    parameter DATA_WIDTH = 32,
    parameter ADDR_WIDTH = 32
)(
    ports
);

    mod1 #(
        .DATA_WIDTH(DATA_WIDTH),
        .ADDR_WIDTH(ADDR_WIDTH)
    )u_mod1(
        ports
    );

    mod2 #(
        .DATA_WIDTH(DATA_WIDTH),
        .ADDR_WIDTH(ADDR_WIDTH)
    )u_mod2(
        ports
    );
endmodule
```

11.6 位宽和维度检查规则

1. 只有多个端口连接到一个wire信号，才会进行位宽和维度检查。
2. 如果多个信号中，conn_name有使用数组索引data[<i>]或者位宽切片data[msb:lsb]，则不会进行维度检查。

12. extFList

12.1 功能简介

extFList(extract file list)用于从包含`ifdefn`/`ifndef`/`else`/`endif`宏的main filelist中，根据添加的define提取出用于实际编译使用的filelist。该功能用于根据选项实现某些子系统（模块）的包空处理，从而减少编译仿真时间。

12. 选项

main filelist：指定main filelist的路径。

-d define：指定一个define，用于控制main filelist的包含或排除。

-o output file：指定输出文件名和目录，默认为.{input_file}_ext.f。

-e：展开filelist中文件路径的环境变量，并把相对路径转换为绝对路径。

-h：显示帮助信息。

```
stk extFList <main filelist> [-d <define>] [f -o <output file>] [-e]
stk extFList main.f -D DUMMY_GPU -o ext.f -e
```

12.2 main filelist填写规则

main filelist是在普通filelist的基础上，添加了``ifdefn`/`ifndef`/`else`/`endif``宏。通过这些宏的条件，选择性地包含或排除文件，得到实际需要编译的filelist。

main.f

```
+incdir+$ENV_ROOT_DIR

`ifndef DUMMY_GPU
    -f $ENV_ROOT_DIR/gpu.f
`else
    $ENV_ROOT_DIR/dummy_gpu.sv
`endif
```

在上面的例子中：

```
stk extFList main.f -D DUMMY_GPU
```

将会生成一个新的filelist，内容如下：

```
+incdir+$ENV_ROOT_DIR

$ENV_ROOT_DIR/dummy_gpu.sv
```

说明：

1. `ifdef和`ifndef支持嵌套
2. main.f中包含的所有子filelist内都可以包含宏，如：gpu.f

13. yaml语法扩展

为了提高yaml的复用性、易用性和扩展性，在基本的yaml语法功能基础上，新增了：`include`，继承，引用节点，引用环境变量等功能

13.1 include

语法：`!include file`

include分为2种形式：

1. 在yaml根节点外使用`!include file`的方式，将其他yaml文件的内容加入到当前文件
2. 在yaml的节点中使用`!include file`，将其他yaml文件的内容，挂载在该节点上

file可以使用环境变量，比如`$$PROJ_ROOT/base.yaml`，`PROJ_ROOT`为环境变量，也可以添加括号`$$ (PROJ_ROOT) /base.yaml`

base.yaml

```
base:
  test: abc
  index: 1

configure:
  times: 10
```

new.yaml

```
!include base.yaml
new:
  node: !include base.yaml
```

展开include得到如下：

new.yaml

```
base:
  test: abc
  index: 1

configure:
  times: 10

new:
  node:
    base:
      test: abc
      index: 1
    configure:
      times: 10
```

13.2 继承

node_a继承parent1,parent2,...

语法：`son_node(parent1,parent2,...):`

- `son_node`：继承者，可以是任意节点
- `parent`：被继承者，必须是根节点

继承后，继承者（`son_node`）会继承被继承者（`parent`）的所有内容，但是如果继承者下有跟被继承者相同名字的节点时有两种处理方法。

1. `override`：即继承者覆盖掉被继承者相同名字的内容
2. `merge`：即将继承者的值与被继承者相同名字的值合并，这种方式需要在继承者名字末尾添加"+"标志。字符串合并时，中间会添加空格。
3. `exclude`：剔除parent中的相关内容

相同名字的节点合并有约束条件：

两个节点的值类型必须相同，且为：`list,string,dict`，否则会报错

base.yaml

```
base:
  str: +define+ABC +define+exclude
  lst:
    - 1
    - 2
  dict:
    addr: Beijin
    age: 18
    num: 10
```

13.2.1 只新增

sub.yaml

```
!include base.yaml
sub(base):
  add: 20
```

等效于

```
sub:
  str: +define+ABC
  lst:
    - 1
    - 2
  dict:
```

```
  addr: Beijin
  age: 18
  add: 20
```

13.2.2 覆盖

```
!include base.yaml
sub(base):
  str: +define+XYZ
  lst: abc
  dict: ijk
  add: 20
```

等效于

```
sub:
  str: +define+XYZ
  lst: abc
  dict: ijk
  num: 10
  add: 20
```

13.2.3 合并

child的内容放在前面，parent的内容放在后面

```
!include base.yaml
sub(base):
  str+: +define+XYZ
  lst+:
    - abc
  dict+:
    name: Jakod
  num+: 60 # 数值类型不允许合并
  add+: 20 # 被继承者没有该子节点，也可以用+
```

等效于

```
sub:
  str: +define+XYZ +define+ABC +define+exclude
  lst:
    - 1
    - 2
    - abc
  dict:
    addr: Beijin
```

```
age: 18
name: Jakod
num: 60 # 不允许合并，采用覆盖的方式
add: 20
```

同名节点值类型不一样，会报错

```
sub(base):
  lst+: ljk # 被继承者的lst值的类型为list,继承者的lst值类型为string，所以报错
```

注意：如果parent下的节点使用了引用，child又使用了合并，可能会出现问题。比如：

```
base:
  names:
    - Candy
    - Mary

parent:
  names: $base.names

child(parent):
  names+:
    - Tom
```

合并的时候认为parent.names为普通的str类型，因为继承的时候还没有处理引用关系，names就是普通的\$base.names，会报与child.names的list类型不匹配无法合并。

13.2.4 剔除

```
!include base.yaml
sub(base):
  str-: +define+exclude
```

等效于

```
sub:
  str: +define+ABC
  lst:
    - 1
    - 2
  dict:
    addr: Beijin
    age: 18
    num: 10
```

13.2.5 合并和剔除

在继承的child中同时使用合并和剔除

```
!include base.yaml
sub(base):
  str+: +define+XYZ
  str-: +define+exclude
```

等效于

```
sub:
  str: +define+XYZ +define+ABC
  lst:
    - 1
    - 2
  dict:
    addr: Beijin
    age: 18
  num: 10
```

13.2.6 自继承

parent和child是同名节点

```
!include base.yaml
base(base):
  str+: +define+XYZ
  str-: +define+exclude
```

等效于

```
base:
  str: +define+XYZ +define+ABC
  lst:
    - 1
    - 2
  dict:
    addr: Beijin
    age: 18
  num: 10
```

注意：child中对同一节点，只能用一次合并和一次剔除，使用多次，只有最后一次生效

注意：自继承只能使用一次，使用多次，只有最后一次生效

推荐：只在字符串类型的情况下使用合并和剔除

覆盖：parent与child类型可以不一样，child直接覆盖掉parent的值

13.3 引用节点

语法：`$node_a.node_b.node_c`或者`$(node_a.node_b[0][2].node_c[3])`

- `node_a,node_b,node_c`为3个连续层级关系的节点名字
- `node_c[3]`表示`node_c`为list，取list的第3个值(从0开始)
- `node_a`的层级必须在引用层级中，或者以外。

编写配置的时候会遇到如下问题

1. 一个内容需要在多个地方重复填写
2. 一个对象的值与其他对象的值有关系（由另外一个对象的值组成）

```
mode: debug
tc: tc001_sanity
seed: {{SEED}}
run_cmd: 'make run mode=$mode tc=$tc seed=$seed'
log: '$mode/log/$(tc)_$(seed).log'
```

在上面的例子中`run_cmd`和`log`两个对象的值都依赖`mode,tc,seed`,通过先定义好依赖对象，然后引用这些对象的方式，非常方便的实现了其他对象值的组成。

为了很好的解决上述问题，在配置文件中通过`$obj_hire`的方式引用另外一个对象。`obj_hire`为引用对象的层次路径，这个路径为向上引用的路径（与SV中的向上层次引用方式完全相同），下面举例说明引用的搜索过程。

```
regr_cfg:
  max_run_num: $num
  num: 10
  lsfs:
    low_mem: $bsub1
    high_mem: $regr_cfg.bsub2
    long:
      - $bsub1
      - $bsub2
  bsub1: 'bsub -R "rusage[mem=1]"'
  bsub2: 'bsub -R "rusage[mem=10]"'
  test: $lsfs

builds:
  build0:
    lsf: $short
  build0:
    lsf: $regr_cfg.lsfs.long[0]
```

1. 在当前层次搜索，搜索引用的第一个节点，如果找到跳转到【3】，没有有找到跳转到【2】
2. 往上一个层次继续搜索第一个节点，如果找到跳转到【3】，每有找到且当前节点不为根节点则跳转到【2】，没有找到且当前搜索节点为根节点，跳转到【5】
3. 进入找到的节点，并在该节点搜索引用后面的节点，如果找到且还有下一级节点则跳转到【3】，如果找到没有下一级节点跳转到【4】，如果没有找到跳转到【2】
4. 引用的对象找到，把该对象的值添加到引用的位置

5. 没有找到引用对象，报错

注意：

如果引用的对象包含引用，那么先把引用对象的值求解出来，再把引用对象的值赋值给到引用位置，如上面的`$regr_cfg.lsfs.long[0]`

- `$num`

在`max_run_num`层级搜索，找到了`num`对象，且该引用只有一个节点，所以引用搜索成功，直接把`num`对象的值拷贝到`max_num_num`对象得到：`max_run_num: 10`

- `$bsub1`

1. 在`low_mem`层级搜索`bsub1`,没有找到`bsub1`对象，进入`lsfs`的层级搜索。
2. 在`lsfs`层级搜索到`bsub1`对象，且该引用只有`bsub`一个节点，所以引用搜索成功，直接把`bsub1`对象的值赋值给`low_mem`对象

- `$regr_cfg.bsub2`

1. 在`low_mem`层级搜索`regr_cfg`,没有找到`regr_cfg`对象，进入`lsfs`的层级搜索。
2. 在`lsfs`层级搜索到`regr_cfg`对象，没有找到`regr_cfg`对象，进入`regr_cfg`的层级搜索。
3. 在`regr_cfg`层级搜索到`regr_cfg`对象，找到`regr_cfg`对象，在`regr_cfg`对象下面继续搜索引用的下一个节点`bsub2`
4. 在`regr_cfg`下面找到`bsub2`,且`bsub2`为引用的最后一个节点，搜索成功，将`busb2`的值赋值给`high_mem`对象

- `$short`

1. 在`lsf`层级搜索`short`,没有找到`short`对象，进入`build0`的层级搜索。
2. 在`build0`层级搜索到`short`对象，没有找到`short`对象，进入`builds`的层级搜索。
3. `builds`层级包含`regr_cfg`和`build0`两个对象，没有搜索到`short`对象，且`builds`为根节点,搜索失败，程序报错。

- `$regr_cfg.lsfs.long[0]`

1. 在`lsf`层级搜索`regr_cfg`,没有找到`regr_cfg`对象，进入`build0`的层级搜索。
2. 在`build0`层级搜索到`regr_cfg`对象，没有找到`regr_cfg`对象，进入`builds`的层级搜索。
3. `builds`层级包含`regr_cfg`和`build0`两个对象，搜索到`regr_cfg`对象，引用还有下一级节点，进入`regr_cfg`搜索`lsfs`
4. 在`regr_cfg`下找到`lsfs`，引用还有下一级节点，进入`lsfs`搜索`long[0]`
5. 在`lsfs`下找到`long`，并判断`long`为数组，搜索成功。
6. `long[0]`值为包含引用，则先把引用的值求解出来，得到`long[0]: 'bsub -R "rusage[mem=1]"'`，然后将`long[0]`的值赋值给`builds.build1.lsf`对象

- `$lsfs`, 这个引用不是`scalar`值，而是`dict`

1. 在`test`层级搜索`lsfs`,没有找到`lsfs`对象，进入`regr_cfg`的层级搜索。
2. 在`regr_cfg`层级搜索到`lsfs`对象，且引用没有下一级节点，搜索成功。
3. 把`regr_cfg.lsfs`对象赋值给`regr_cfg.test`对象

注意

1. 禁止循环引用，这样会导致程序死循环。比如`bsub1: $bsub2; bsub2: $bsub1`
2. 如果引用与其他字符串有拼接，且会产生歧义，需要对引号变量使用括号包裹

```
log: $mode/log/$name_$seed.log # 本意为$name + '_'会被错误解析为\$(name_)
## 13. 应该修改为
log: $(mode)/log/$(name)_$(seed).log
```

13.3 引用环境变量

语法：`$$ENV_VAR`或者`$(ENV_VAR)`, `ENV_VAR`为环境变量名字

在工程中，为了提高环境的可移植性，会把一些路径定义为环境变量，如果配置的对象依赖环境变量，可以通过`$$ENV_VAR`或者`$(ENV_VAR)`的方式引用环境变量。

回归工具解析到对象的值包含环境变量时会把`$$ENV_VAR`或`$(ENV_VAR)`替换为环境变量的实际值

```
$SIM_ROOT_DIR=/home/env_root
```

```
mode: base_fun
tc: tc001_saniy
log: $$SIM_ROOT_DIR/$mode/log/$(tc)_$(seed).log
```

log=/home/env_root/base_fun/log/tc001_sanity_xxx.log

注意

~无法解析为user的home目录，因为程序无法知道字符串是否为路径

13.4 SEED

在回归过程中，一个用例执行多次，每次需要提供不同的种子，为了实现该功能，定义了一种特殊写法`{{SEED}}`，脚本每回归一条用例会把其被替换一个随机的8位10进制数。

13.5 保留\$

如果要保留\$可以通过转义的方式`\$`，解析后会转为\$

13.6 date

在定义输出文件名的时候，有的时候需要日期作为文件名的一部分，regress的yaml中默认会添加一个`date`根节点，值为解析yaml的时刻

```
yyyymmddhhmmss
```

13.7 扩展类型

标准yaml有dict/list/scalar几种数据类型，为了使用方便，从这些类型中定义出如下类型：
且对象的值为如下几种类型：

1. scalar

某个对象的值必须为纯量（string,数值等，不能为dict或者list），某些对象会对纯量值的类型有约束，比如`max_run_num`必须为整数类型

2. list-scalar

某个对象的值可以为纯量或者数组，但是数组的成员必须为纯量。

无论对象的值是否是数组，获取对象的值都会得到一个数组

- 对象值为list：则直接返回该list
- 对象值为scalar：则返回[scalar]

3. join_list-scalar

某个对象的值可以为纯量或者数组，但是数组的成员必须为纯量。

无论对象的值是否是数组，获取对象的值都会得到一个纯量 (string)

- 对象值为scalar：则直接返回scalar
- 对象值为list：则返回' '.join(list)

4. list-join_list-scalar

某个对象的值可以为纯量或者数组，数组的值可以为纯量，也可以为数组，但是二级数组的值必须为纯量。

无论对象的值是纯量，一维数组还是二维数组，获取该对象的值都会得到一个一维数值。

- 对象值为纯量：返回[scalar]
- 对象值为list：返回该list
- 对象为list-list：返回[' '.join(x) for x in list-list]，即数组的第二维通过join的方式合并成一个值，第二维允许是稀疏的。如下：

```
list-join_list-scalar:
- abc
-
- xyz
- www
```